SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT-1

QUESTIONS

CO1: Apply classical encryption techniques using symmetric and asymmetric ciphers to encrypt and decrypt data for the ensuring data security.

Assignment Title: Evaluate, Design and Defend a Secure Patient-Record Exchange Strategy for a Multi-Hospital Healthcare Network (Total Marks: 50)

Code of Conduct:

I **P.Sunayana(192424303)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Introduction, Scope and Method

a. The scenario in one paragraph

The regional network links one central hospital with 25 satellite clinics. Clinics send patient records, diagnostic reports, prescriptions and billing data to the central hospital over a public network and receive specialist opinions in return. Personnel hold five clearance levels: Medical Director (Top Secret), Specialist Doctors (Secret), Nurses and Lab Staff (Confidential), Billing and Front-Desk Staff (Restricted) and Patients (Public). The system must keep diagnoses confidential, detect tampering with prescriptions, dosage and lab values, stop staff from reading.

b. What the assignment really asks

The assignment is deliberately written with the verbs evaluate, judge, defend and re-evaluate. It is therefore not a request to copy textbook descriptions of Caesar or Vigenère. It asks four deeper questions, which structure the whole report:

How strong is each technique, measured rather than asserted? Each cipher is encrypted with clinical notes of 100 to 100,000 letters, attacked with a technique suited to it (frequency analysis, Kasiski examination, index of coincidence, hill climbing, known-plaintext linear algebra), and ranked by the evidence.

What do classical techniques fail to provide? The report separates confidentiality (which classical ciphers partly deliver) from integrity, authentication, non-repudiation and availability (which they do not), and demonstrates each gap with a working exploit.

Does the access-control policy hold when reality intrudes? Bell-LaPadula is applied to 20 concrete requests, including emergencies, declassification and a doctor who must write to a lower-level billing file.

What happens when the assumptions break? On Day 4 an insider copies hundreds of ciphertexts under one key, many starting with predictable headers. The design is re-tested, then retained, modified or redesigned.

c. Method and environment

All work was done in Python 3 with NumPy, SciPy, Matplotlib and Pillow. The code is organised as one library (cryptolab.py) that contains every cipher, attack, access-control monitor and steganography routine, plus one script per task group (s1_design.py to s7_day4.py) and a unit-test suite (tests.py). Each script writes a text log, a JSON result file and its figures.

The "execution screens" in this report are rendered directly from those logs, so every number printed in a screen is the number the program produced. The experiments are fully reproducible because every random choice uses a documented seed.

Table 1: Experimental environment and fixed parameters

Item	Value
Language / libraries	Python 3, NumPy, SciPy (chi-square p-values), Matplotlib, Pillow (JPEG tests)
Clinical text	Synthetic notes generated from 7 sentence templates and about 120 clinical phrases (gen_notes); letters only after cleaning
Attacker reference corpus	About 500,000 letters of general English (Python documentation) plus 60,000 letters of public-style clinical text (seed 99, different from the test notes)
Caesar key	k = 11
Monoalphabetic key	random permutation from seed 2024
Vigenère key	QZMKTWBRLXHD (12 letters)
Columnar key	MEDICAL (7 columns, order 7-4-3-5-2-1-6)
Hill keys	K26 = [[6,24,1],[13,16,10],[20,17,15]] over Z26; K36 = [[5,7,6],[9,5,9],[2,7,8]] over Z36
Image	512×512 synthetic CT-like phantom (Shepp-Logan style) with Gaussian noise, 8 bit
Message sizes	100, 1,000, 10,000 and 100,000 letters of identical plaintext for timing and tests; 200 to 10,000 for cryptanalysis

Ethics and data protection
No real patient data, real hospital names or real credentials are used anywhere. Names such as "Dr Rao" and identifiers such as PT20481 are invented. The attack code is educational, operates only on synthetic data, and breaks ciphers that are in any case obsolete.

d. Notation

Table 2: Notation used throughout the report

Symbol	Meaning
P, C, K	plaintext, ciphertext, key
Z_m	integers modulo m (m = 26 for letters, 36 for letters plus digits, 256 for image intensities)
IC	index of coincidence: IC = Σ f_i (f_i - 1) / (N (N - 1)); English ≈ 0.066-0.070, uniformly random letters ≈ 0.0385
GL_n(Z_m)	group of invertible n×n matrices with entries modulo m; |GL_n| is the Hill key-space size
L × I	risk score = likelihood (1-5) multiplied by impact (1-5)
BLP, ss, *	Bell-LaPadula model, simple security property (no read up), star property (no write down)
KPA, CPA	known-plaintext attack, chosen-plaintext attack

e. Key results at a glance

Because the report is long, the headline results are collected here, each with the chapter that contains the evidence. They are the numbers the rest of the report will explain and defend.

Table 3: Headline results

Question	Result	Where
Largest key space among single schemes	Monoalphabetic: 26! = 4.03×10^26 keys (88.4 bits)	Chapter 6
...and how fast it falls	Plaintext fully recovered; success rate ≥90% from 400 letters, 0.07 s for 1,000 letters	Chapter 7
Vigenère, 12-letter key	IC peaks at m = 12 (0.068 vs 0.040 elsewhere); ≥90% success from 800 letters	Chapter 7
Columnar transposition, 7 columns	Only 5040 keys; broken at every length tested down to 50 letters	Chapter 7
Hill 3x3 (ciphertext IC)	IC = 0.0390 (random is 0.0385) but key recovered from 15 known letters in 100% of cases for header text	Chapters 6-7
Product (substitution + transposition)	Needs about 400 letters for ≥90% success: stronger than either part alone	Chapter 8
Dosage tampering without the key	AMOX500MG became AMOX900MG undetected	Chapter 3
Unkeyed checksum	Row swap undetected by additive checksum; compensating edit defeats both checksums	Chapter 8
Image shift cipher	Entropy and neighbour mutual information unchanged: complete structure leak; key from one pixel pair	Chapter 8
LSB steganography	PSNR 76.4 dB for a 136-byte report; every JPEG quality destroys it	Chapter 9
Bell-LaPadula	20 requests decided: 9 denied, 1 break-glass, 2 trusted-subject actions	Chapter 4

f. Glossary

Table 4: Terms used in the report

Term	Meaning
Keyspace	The set of all possible keys of a scheme; its size is the work needed for exhaustive search.
Unicity distance	The length of ciphertext after which only one meaningful decryption remains; U = H(K) / D where D is the redundancy of the language in bits per letter.
Diffusion	Spreading the influence of one plaintext symbol over many ciphertext symbols (Hill and transposition).
Confusion	Making the relation between key and ciphertext complex (substitution).
Depth	Several messages enciphered with the same key and aligned in position, which allows statistical attack column by column.
Known-plaintext attack	The attacker holds some plaintext/ciphertext pairs, such as a predictable header.
LSB	Least significant bit of a pixel; its modification changes grey level by at most one.
PSNR	Peak signal-to-noise ratio between two images, in dB; above 40 dB differences are normally invisible.
Chi-square (pairs of values)	Test that detects LSB embedding because it equalises the counts of values 2k and 2k+1.
Break-glass	An emergency override of access control that is allowed but fully audited.
Trusted subject	A process permitted to violate the star property because it is certified not to leak.

g. Road map

Chapter 2 (Task 1) identifies and ranks the attacks. Chapter 3 (Task 2) maps security goals to system functions. Chapter 4 (Task 3) applies Bell-LaPadula. Chapters 5-13 design the ciphers, analyse their mathematics, attack them, evaluate transposition and product ciphers and steganography, present the implementation and test evidence and the experimental results, recommend the final design, and carry out the Day-4 redesign. Chapter 14 concludes. The appendices contain the complete source code and execution logs.

2 Task 1 - Evaluate Threats, Attacks and Risks

a. Four terms that are often confused

A security analysis fails quickly when the vocabulary is loose. In this report threat, vulnerability, attack and risk are used in the following precise senses, each illustrated from the healthcare scenario.

Table 5: Threat, vulnerability, attack and risk distinguished

Term	Definition	Example from this scenario
Threat	A potential cause of harm: an actor or event with the capability and motive to damage an asset.	A malicious clinic employee, an organised-crime group selling medical identities, or a competitor wishing to disrupt the network. The threat exists whether or not anything has happened.
Vulnerability	A weakness in the system, process or people that a threat could exploit.	The public network carries records without authentication; one shared key is used at all 25 clinics; record headers are predictable; front-desk staff are minimally trained.
Attack	A deliberate action that exploits a vulnerability to realise a threat.	Changing "500 mg" to "900 mg" in a ciphertext in transit; replaying yesterday's prescription; copying months of ciphertext and cryptanalysing it.
Risk	The expected harm: likelihood of an attack succeeding multiplied by its impact on the asset.	Dosage tampering: likelihood 4/5 (no integrity control exists) × impact 5/5 (patient harm) = 20/25. Risk is a property of the *pair* threat-vulnerability, not of the attacker alone.

A useful way to hold these together is the chain: a threat actor uses an attack to exploit a vulnerability and the risk is the product of how likely this is and how much it would hurt. Removing a vulnerability reduces risk without needing to know who the attacker is, which is why the later design work concentrates on vulnerabilities (shared keys, predictable headers, no authentication) rather than on attackers.

b. Assets and attacker models

The assets are, in decreasing order of what a mistake costs: (i) the correctness of prescriptions, dosages and lab values, because errors can injure or kill a patient; (ii) the confidentiality of diagnoses, especially for stigmatised conditions such as HIV infection, cancer and mental illness; (iii) the availability of the records system during clinical hours; and (iv) the accountability of who ordered what. Two attacker models are used. The external attacker controls the public network between clinic and hospital: he can read, delay, drop, copy, insert and modify messages (the classical Dolev-Yao capabilities). The insider is a legitimate employee of one clinic who holds that clinic's credentials and equipment. The insider model is deliberately held back for Task 12, where it changes the ranking.

c. Catalogue of attacks

Seven attacks are analysed, one more than the minimum of six. For each one the classification as passive (observes) or active (alters, injects or disrupts) is justified, the vulnerability it exploits is named, and its consequence is described in terms of both privacy and patient safety.

A1 - Eavesdropping on record upload (passive)

The public network lets any party on the path copy packets. A passive attacker never changes anything, which is exactly why it is hard to detect: the clinic and hospital continue to work normally while ciphertexts are silently archived. Because rural clinics typically use shared or poorly protected links (mobile data, community Wi-Fi), the likelihood is rated 5/5. The impact is a privacy breach of diagnoses (impact 4/5 for privacy, only 1/5 for safety because no clinical data is changed). Classical ciphers are the only defence in this task, so the strength of the cipher decides the risk; it is the subject of Tasks 4-6.

A2 - Traffic analysis (passive)

Even if every payload is perfectly encrypted, the attacker learns who communicates with whom, when and how much. A sudden burst of traffic from the oncology specialist at the central hospital to one village clinic reveals that a villager has been diagnosed with cancer. Message length is also a side channel: the length of a ciphertext is the length of the plaintext for every classical cipher in this report. Likelihood 4/5 (cheap to perform), privacy impact 2/5, safety impact 1/5. Padding messages to fixed lengths and sending dummy traffic reduce it, but no cipher removes it.

A3 - Modification of prescriptions, dosages and lab values (active)

An attacker, or a corrupted forwarding node, alters data in transit. Section 3.4 shows that a Vigenère or Caesar ciphertext can be edited without the key so that the plaintext changes predictably (a digit moves from 5 to 9), and that a transposition cipher lets whole rows be swapped. Likelihood 4/5 because no classical technique offers integrity, so any attacker who can intercept can also modify. Impact 5/5 for safety: a tenfold insulin dose or a wrongly flagged potassium value can kill. Privacy impact is only 1/5, which is why this attack is under-weighted by purely privacy-oriented risk assessments.

A4 - Masquerade as a doctor (active)

An attacker, or a nurse at a clinic, sends a message that claims to come from a specialist, for example an order to change a medication. When every clinic shares one key (the simplest design for untrained staff), successful decryption proves nothing about the sender

(demonstrated in Section 3.5). Likelihood 3/5 because the attacker needs the key or access to a clinic terminal; impact 5/5 for safety (false orders) and 4/5 for privacy (impersonation to extract records).

A5 - Replay of old orders (active)

The attacker records a legitimate encrypted order and re-sends it later. If the message has no timestamp, sequence number or nonce, the receiver cannot tell old from new: a "give 10 units of insulin" instruction is executed twice. Likelihood 3/5, safety impact 4/5. Classical ciphers are deterministic for a given key and plaintext, so they actively help replay: identical plaintexts yield identical ciphertexts.

A6 - Denial of service on the records system (active)

Flooding the central server, cutting the network link or corrupting messages so that decryption fails prevents clinicians from reaching records during an emergency. This is the one attack where encryption plays almost no role. Likelihood 3/5, impact 4/5 (delayed care, paper fall-back), privacy impact 1/5. Mitigation belongs to network engineering: redundant links, rate limiting, offline copies of critical data at each clinic.

A7 - Insider cryptanalysis and key recovery (passive turning active)

A clinic employee copies ciphertexts over months, then attacks the cipher offline, using predictable headers as known plaintext. Before the Day-4 incident this is rated likelihood 2/5 (it needs patience and some skill) but impact 5/5 for privacy: the recovered key unlocks every message at all 25 sites if one network-wide key is used. After the incident the likelihood becomes 5/5 (Section 13.1). It starts as a passive attack (copying) and becomes active once the key is recovered and forged messages are sent.

d. Passive versus active classification

Table 6: Classification of the attacks

ID	Attack	Class	Violates	Detectable by receiver?
A1	Eavesdropping	Passive	Confidentiality	No - leaves no trace
A2	Traffic analysis	Passive	Confidentiality (metadata)	No
A3	Modification	Active	Integrity	Only if an integrity check exists
A4	Masquerade	Active	Authentication, non-repudiation	Only if sender authentication exists
A5	Replay	Active	Integrity (freshness)	Only with nonce/timestamp
A6	Denial of service	Active	Availability	Yes - but too late

ID	Attack	Class	Violates	Detectable by receiver?
A7	Insider cryptanalysis	Passive → Active	All of the above, via key loss	No until forged traffic appears

The table shows an asymmetry that drives the whole design. Passive attacks are undetectable and must be prevented, which is the job of encryption. Active attacks are detectable in principle but only if the design includes redundancy and authentication, which classical ciphers do not supply. A system built only from classical ciphers is therefore strong against none of the active attacks and, as Tasks 5-6 will show, only conditionally strong against the passive ones.

e. Risk rating: likelihood × impact

Likelihood is rated on a five-point scale (1 = rare, 5 = almost certain) from the attacker's required skill, access and cost, and from whether the existing controls would stop him. Impact is rated 1-5 separately for privacy and for patient safety, and the final impact is the larger of the two. Using the maximum rather than the average is a deliberate choice: an attack that cannot hurt privacy but can kill a patient must not be diluted by averaging. The resulting score L × I ranges from 1 to 25 and is banded green (≤5), yellow (6-10), orange (11-15) and red (≥16).

Table 7: Risk table (before the insider incident)

Rank	ID	Attack	Class	L	I priv	I safety	I	L×I
1	A3	Modification of prescription / dosage / lab value	Active	4	1	5	5	20
2	A1	Eavesdropping on record upload	Passive	5	4	1	4	20
3	A4	Masquerade as a doctor	Active	3	4	5	5	15
4	A5	Replay of an old order	Active	3	1	4	4	12
5	A6	Denial of service on the records system	Active	3	1	4	4	12
6	A7	Insider cryptanalysis / key recovery	Passive->Active	2	5	4	5	10
7	A2	Traffic analysis (who sends what, when)	Passive	4	2	1	2	8

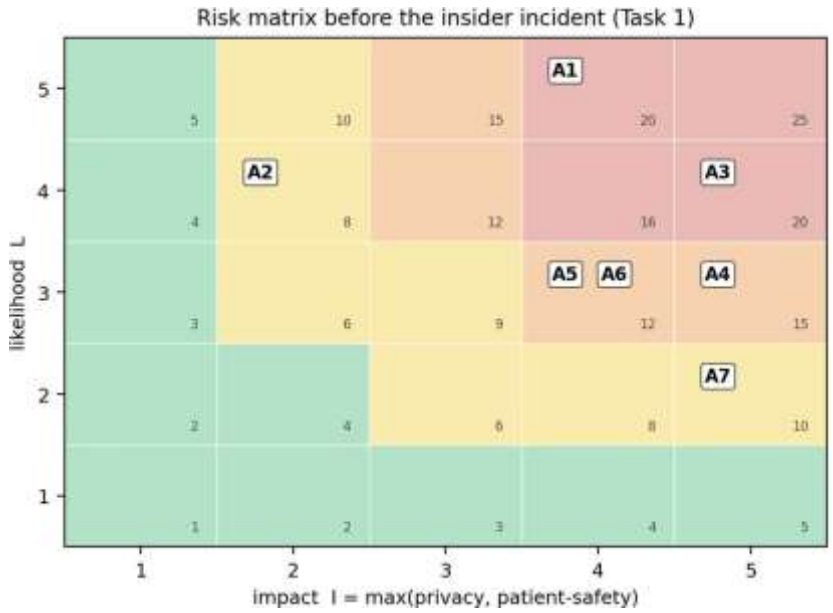


Figure 1: Risk matrix: letters mark attack positions (likelihood vertical, impact horizontal); cell numbers are L × I.

Why these ratings

- **A1 and A3 tie at 20.** Eavesdropping is almost certain (L = 5) with serious privacy impact (4); modification is slightly less certain (L = 4) because the attacker must also know *where* in the ciphertext to act, but its impact is the maximum (5). The tie is broken by patient safety: A3 has I(safety) = 5, A1 only 1.
- **A4 masquerade scores 15.** It needs credentials, so L = 3, but its impact is maximal on both dimensions.
- **A5 and A6 tie at 12.** Replay and denial of service both reach safety impact 4. They are equal in this table; replay is treated first because it is cheap to fix with a sequence number, whereas denial of service needs redundant infrastructure.
- **A7 scores only 10 before Day 4.** The rating reflects what the designers believed. The insider incident shows this belief was wrong; Task 12 re-rates it to 25, the highest score in the table.
- **A2 is lowest at 8** because its direct harm is small, but it is the attack no cipher in this report can prevent.

f. Defending the ranking: what is addressed first, and why

Priority 1: A3, modification of prescriptions, dosages and lab values. Three arguments support this choice even though eavesdropping has the same numerical score.

Irreversibility. A privacy breach is serious but bounded: the information is out, and the remedy is notification and compensation. A wrong dosage administered to a patient is unbounded and cannot be undone.

i. **Detection asymmetry.** Eavesdropping cannot be detected, but modification *can* be detected cheaply once redundancy is added. Controls against A3 are therefore the best "return on design effort", and they also reduce A4 and A5 because authentication and freshness are similar mechanisms.

ii. **Ethical weighting.** Medical ethics ranks non-maleficence ("first, do no harm") ahead of confidentiality when the two compete. A system that protects secrecy but allows silent tampering has the order of priorities backwards.

Priority 2: A1 eavesdropping, handled by the strongest cipher the classical toolkit allows, together with the access-control policy that decides who may receive which plaintext. Priority 3: A4 and A5, handled by sender-specific keys and sequence numbers

g. Sensitivity of the ranking to the impact weighting

A ranking is only defensible if it does not flip when a reasonable reviewer changes the weights. Three alternative impact definitions were applied to the same likelihood values: privacy-only impact and the mean of the two.

Table 8: Rank of each attack under four impact definitions

ID	Attack	Privacy-only	Safety-only	Mean	Max (used)
A1	Eavesdropping on record upload	1 (20)	6 (5)	2 (12.5)	1 (20)
A2	Traffic analysis (who sends what, when)	4 (8)	7 (4)	7 (6.0)	7 (8)
A3	Modification of prescription / dosage / lab value	5 (4)	1 (20)	3 (12.0)	1 (20)
A4	Masquerade as a doctor	2 (12)	2 (15)	1 (13.5)	3 (15)
A5	Replay of an old order	6 (3)	3 (12)	5 (7.5)	4 (12)
A6	Denial of service on the records system	6 (3)	3 (12)	5 (7.5)	4 (12)
A7	Insider cryptanalysis / key recovery	3 (10)	5 (8)	4 (9.0)	6 (10)

Three observations follow. (1) A1 eavesdropping is first under the privacy-only view and A3 modification is first under the safety-only view, which is exactly the tension identified in Section 2.6; the max rule, which the report uses, places them jointly first and resolves the tie toward safety. (2) A2 traffic analysis is never above rank 6 under any weighting, so leaving it as a residual risk is robust. (3) A4 masquerade stays in the top three under every definition, so

any design that leaves sender authentication unsolved is not defensible whichever weighting the reviewer prefers. The conclusion that integrity and authentication controls should be designed before the finest confidentiality tuning therefore survives the sensitivity test.

h. From attack to control: what classical techniques can contribute

Table 9: Mitigation map for the seven attacks

ID	Possible control	Classical technique available?	Residual risk
A1	Encrypt all records and reports	Yes - strongest classical layering (Chapters 7-8, 12)	Cipher still breakable with enough ciphertext; key distribution to 25 sites
A2	Fixed-length padding, dummy traffic	Partly - padding with nulls is classical	Timing and volume metadata remain
A3	Redundancy, checksum, check digits	Only weak checksums (Section 8.4); defeated by compensating edits	High - needs a MAC or digital signature
A4	Per-sender keys, call-back verification	Per-clinic key books only; no per-person authentication	High - shared secret cannot identify an individual
A5	Sequence numbers inside the plaintext, daily key change	Yes, as a protocol rule using classical ciphers	Medium - counters must be kept in sync at 25 sites
A6	Redundant links, offline emergency summary at the clinic	No - infrastructure issue	Medium
A7	Per-message keys, key rotation, one-time pads for critical fields	Yes - the heart of the Day-4 redesign (Chapter 13)	Low for short fields (pad), medium for bulk text

i. Assumptions and boundaries of this analysis

- **Likelihood is judgemental.** The ratings are structured expert judgements, not measured frequencies; no public statistics exist for classical-cipher attacks on rural clinic networks. The sensitivity analysis above is the safeguard against over-reliance on any single number.
- **The network is hostile, the endpoints are not (yet).** For Tasks 1-11 the clinic terminals are assumed to be honest and physically controlled; Task 12 relaxes this assumption.
- **Impact is per incident.** Scaling by the number of patients affected would raise the privacy impact of A1 and A7 (one key opens 25 sites) well above their single-incident ratings.
- **Only technical controls are ranked.** Training, physical security and legal deterrence are important but outside CO1.

Evidence for Task 1
The risk table, the risk matrix and the three-part defence of the priority order (Section 2.6). The ranking is revisited with the Day-4 evidence in Section 13.1 and Figure "Risk matrix after the incident".

3 Task 2 - Map and Judge Security Goals

a. The five goals

Table 10: Security goals and what they mean in the clinic

Goal	Question it answers	Clinical example
Confidentiality	Can anyone unauthorised read it?	A village neighbour cannot learn a patient's HIV status.
Integrity	Has it been changed since the sender produced it?	The lab value 5.1 mmol/L potassium is not silently turned into 3.1.
Availability	Can authorised users get it when needed?	The doctor can open the allergy record at 3 a.m.
Authentication	Is the claimed sender really the sender?	The order "stop anticoagulants" really comes from the treating specialist.
Non-repudiation	Can the sender later deny sending it?	A doctor cannot deny having prescribed a dose that harmed a patient; the hospital cannot be falsely accused either.

b. Which goals matter most for each function

The three system functions have different goal profiles. Importance is scored 1-5 and shown in the heat map below (the lower strip shows what classical ciphers deliver, scored on the same scale).

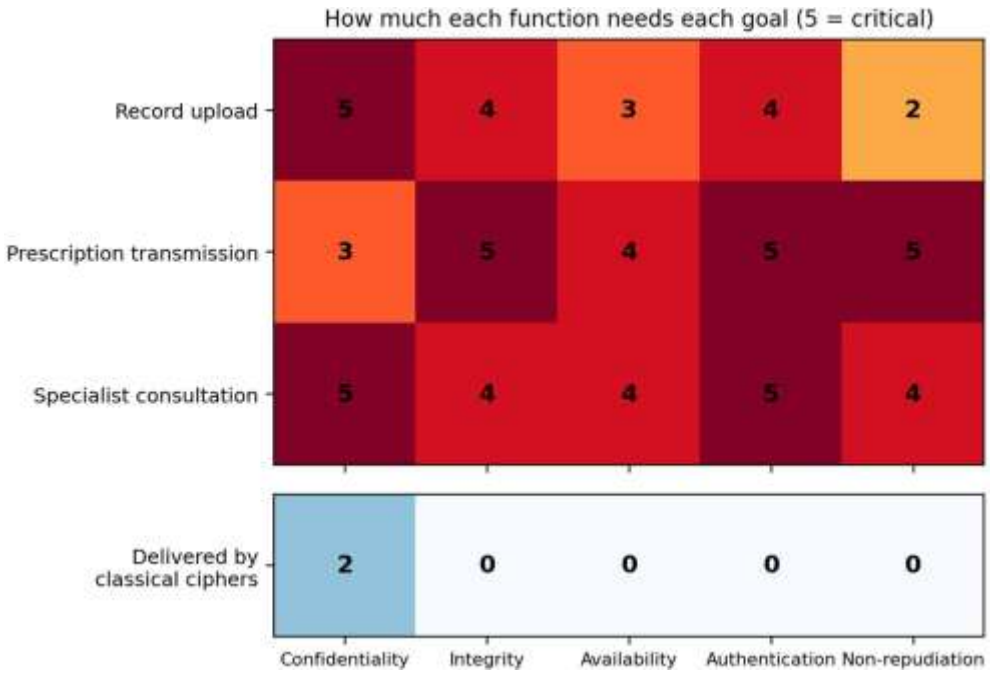


Figure 2: Goal importance per function (top) versus what classical ciphers deliver (bottom).

Record upload (clinic to central hospital)

Records here are bulk and mostly archival, containing diagnoses, histories and billing data. Confidentiality (5) dominates because the bulk of sensitive information is concentrated here. Authentication (4) is the second need: the hospital must be sure that the record belongs to the clinic that claims to have sent it, otherwise a false record could enter a patient's file. Integrity (4) follows because the record becomes the clinical basis for later decisions. Availability (3) is moderate: uploads can be queued and retried. Non-repudiation (2) matters little, since the record is a statement of the clinic, not a legal order.

Prescription transmission

A prescription is short, structured and consequential. Integrity (5) and authentication (5) are absolute: a prescription is useful only if the dosage is exactly as written and the prescriber is who he claims to be. Non-repudiation (5) is also critical because prescriptions are legal instruments, and the prescriber must not be able to deny them nor be framed by a forger. Availability (4) matters because a patient may be waiting at the pharmacy counter. Confidentiality (3) is lower than for records but not trivial: the drug name often reveals the diagnosis (an antiretroviral reveals HIV).

Specialist consultation (opinion from the central hospital)

Specialist opinions are the clinical decision itself. Confidentiality (5), authentication (5) and integrity (4) carry the most weight, with non-repudiation (4) and availability (4) next because consultation is often urgent. The reverse direction is as important as the forward direction: a clinic must be able to prove to an auditor that a treatment followed a specialist's written opinion.

c. What classical ciphers can and cannot deliver

Table 11: Goal delivery by classical techniques (judgement)

Goal	Can classical techniques deliver it?	Verdict and evidence
Confidentiality	Partly, and only against weak attackers	Caesar, monoalphabetic and short-key Vigenère fall in milliseconds to a few seconds (Chapter 7). A Hill cipher resists ciphertext-only frequency analysis but not known-plaintext. Key reuse destroys all of them (Chapter 13). Score 2/5.
Integrity	No	Ciphertexts are malleable. A decrypted message that "looks valid" is not evidence of integrity (Sections 3.4 and 8.4). A checksum can be added, but unkeyed checksums are recomputable and keyed linear checksums are recoverable. Score 0-1/5.
Availability	No	Encryption adds nothing to availability and can reduce it: a lost key makes all data unreadable. Score 0/5.

Goal	Can classical techniques deliver it?	Verdict and evidence
Authentication	No	Symmetric ciphers with a shared key authenticate nobody in particular; every key holder can forge. Score 0/5.
Non-repudiation	No	Requires a secret known only to the signer plus a way for a third party to verify without the secret, which means asymmetric signatures. A shared-key cipher cannot provide it by construction. Score 0/5.

The central conclusion of this chapter is that the classical toolkit addresses one of the five goals, and that goal only weakly. The goals the system needs most for prescriptions (integrity, authentication, non-repudiation) are exactly the ones classical techniques cannot deliver. This is not a defect of implementation; it is structural, as the next two sections show by experiment.

d. Why a correctly decrypted message does not prove its dosage is unaltered

Decryption is a function: given any ciphertext of the right length it returns some plaintext. For the byte-wise additive ciphers (Caesar, Vigenère, one-time pad) the relation is linear: if the attacker adds δ to ciphertext symbol i, the receiver's plaintext symbol i changes by exactly δ, and the key cancels out. The attacker needs to know no key material; he needs only to know which symbol to change and by how much, which in a structured field such as "AMOX500MG" is easy to guess.

$$C'_i = C_i + \delta \implies P'_i = C'_i - K_i = P_i + \delta$$

Demonstration 2a used a Vigenère cipher over the 36-symbol alphabet with the key RXQ7MZ. The plaintext AMOX500MG was enciphered to R94UHPH9W. The attacker added +4 to the fifth ciphertext symbol (H became L), turning the plaintext digit 5 into 9. The receiver decrypted the result to AMOX900MG, with no error, no warning and no way to know that anything had happened. A 500 mg dose of amoxicillin had silently become 900 mg. Because the receiver has no redundancy to check against, "the message decrypted to something sensible" carries no evidential weight whatsoever.

Demonstration 2b repeated the experiment with the Hill cipher. A change of one ciphertext symbol corrupted the entire three-symbol block (AMOX500MG became AMONP30MG), which is a mild form of diffusion. This increases the attacker's difficulty in making a specific change, but he can still destroy data at will, and the receiver still receives no signal that tampering occurred; the garbage may even be mistaken for a typing error.

e. Why a correctly decrypted message does not prove who sent it

When all 25 clinics and the central hospital share the same key (the easiest scheme to administer), possession of the key is the only credential. Demonstration 2c shows an insider at clinic 12 enciphering the text FROMDRRAOAMOX500MG. The central hospital decrypts it successfully and has no means of telling whether Dr Rao or the insider wrote it. Even with a separate key per clinic the receiver learns only which clinic, not which doctor, and every employee of that clinic can still impersonate every other. Authentication in the cryptographic sense requires that only the claimed sender could have produced the message; a secret shared between two or more people cannot provide this, and symmetric classical ciphers have nothing else.

Demonstration 2d shows that a ciphertext accepted on day 1 is accepted again, bit for bit, on day 9. Without a nonce, counter or timestamp inside the encrypted plaintext, replayed orders are indistinguishable from fresh ones.

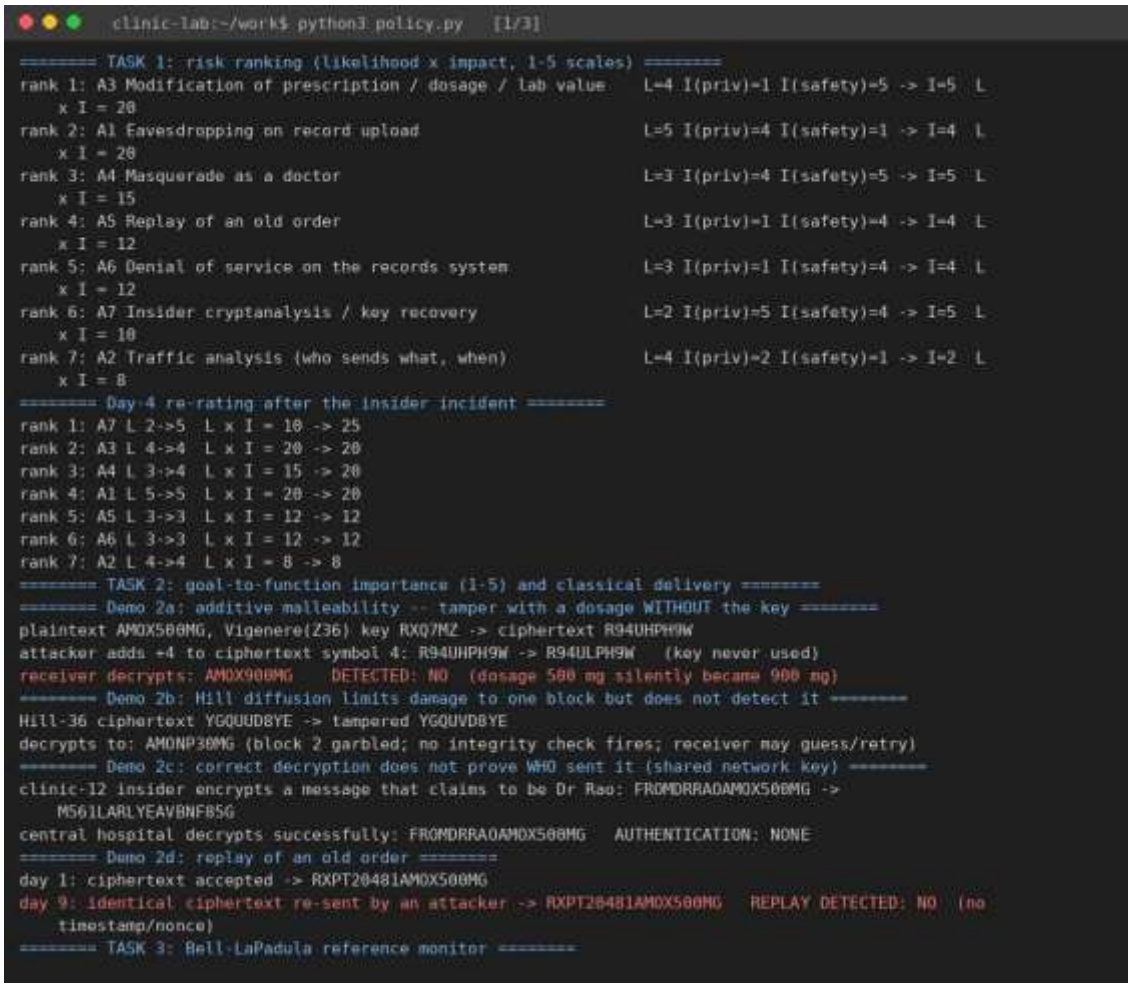


Figure 3: Execution output of s3_policy.py: risk ranking, goal demonstrations 2a-2d and the Bell-LaPadula monitor (screen 1)

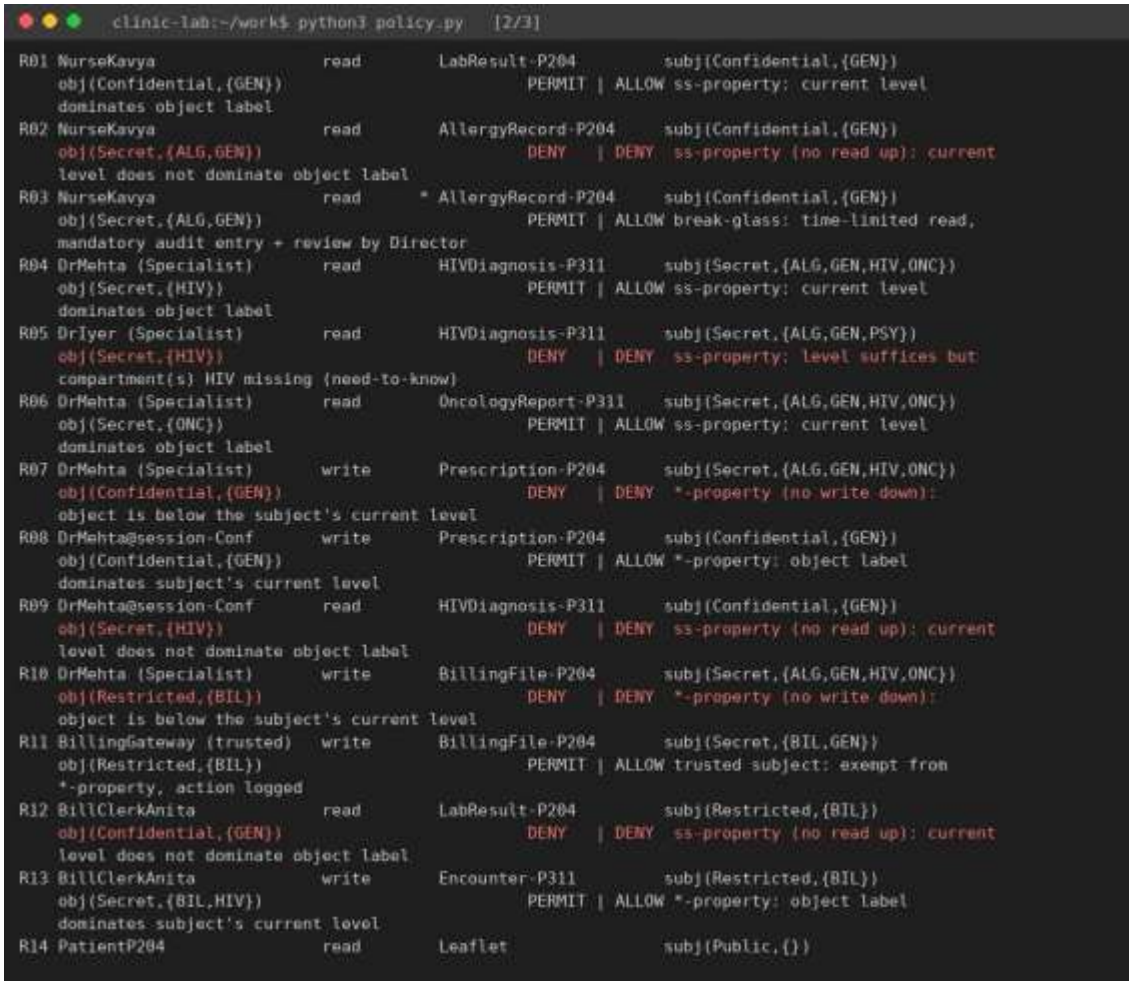


Figure 4: Execution output of s3_policy.py: risk ranking, goal demonstrations 2a-2d and the Bell-LaPadula monitor (screen 2)

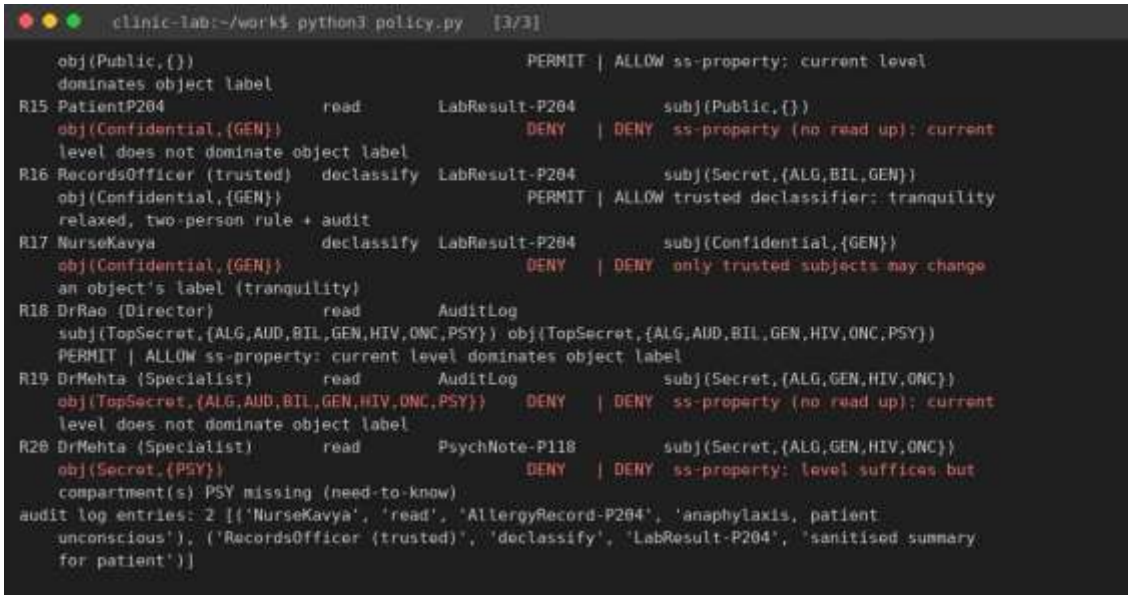


Figure 5: Execution output of s3_policy.py: risk ranking, goal demonstrations 2a-2d and the Bell-LaPadula monitor (screen 3)

f. Linking attacks to goals

Chapter 2 listed attacks and this chapter lists goals; the two are two views of the same problem. Every attack is the negation of at least one goal, and the table below shows which function suffers most, so that the design effort can be directed at the intersections that matter most.

Table 12: Attack-to-goal-to-function map

Attack	Goal negated	Function most exposed	Why
A1 eavesdropping	Confidentiality	Record upload (bulk, rich in diagnoses)	Highest volume of sensitive data per message
A2 traffic analysis	Confidentiality of metadata	Specialist consultation	Pattern of referrals reveals the condition
A3 modification	Integrity	Prescription transmission	Single character changes dosage
A4 masquerade	Authentication	Prescription and consultation	False orders and false opinions
A4 + denial by sender	Non-repudiation	Prescription transmission	Prescriber may deny a harmful order or be framed
A5 replay	Integrity (freshness)	Prescription transmission	Old order executed twice
A6 denial of service	Availability	All three; worst for consultation	Urgent opinions delayed
A7 insider	All	All	Key loss ends every guarantee at once

g. Judgement

Evidence for Task 2

Goal-importance heat map, goal-delivery table and three working exploits: dosage malleability (2a), unauthenticated forgery (2c) and replay (2d). Conclusion: classical ciphers deliver weak confidentiality only; integrity, authentication, non-repudiation and availability must come from other mechanisms.

4 Task 3 - Apply and Critique the Bell-LaPadula Model

a. The model in brief

Bell-LaPadula (BLP, 1973) is a formal model of confidentiality in multilevel secure systems. Every subject (user or process) has a clearance and every object (record) has a classification, both expressed as a label (level, set of compartments). Label A dominates label B when A's level is at least B's level and A's compartment set contains B's. The model has two mandatory rules and one discretionary rule:

- **Simple security property (ss, "no read up"):** a subject may read an object only if the subject's current label dominates the object's label.
- **Star property (*, "no write down"):** a subject may write (append) to an object only if the object's label dominates the subject's current label. This stops a Secret-cleared subject from copying Secret data into a Public file, even by accident or through malware.
- **Discretionary security property (ds):** access must also be permitted by an access-control list (need to know).

Two further notions complete the picture. Tranquility requires that labels do not change while the system is in use (strong tranquility) or at least not in a way that violates the policy (weak tranquility). Trusted subjects are processes exempted from the star property because the system could not operate otherwise, for example a downgrader that releases sanitised reports to a lower level; they are therefore security-critical and must be small and audited.

b. Security levels and compartments for the network

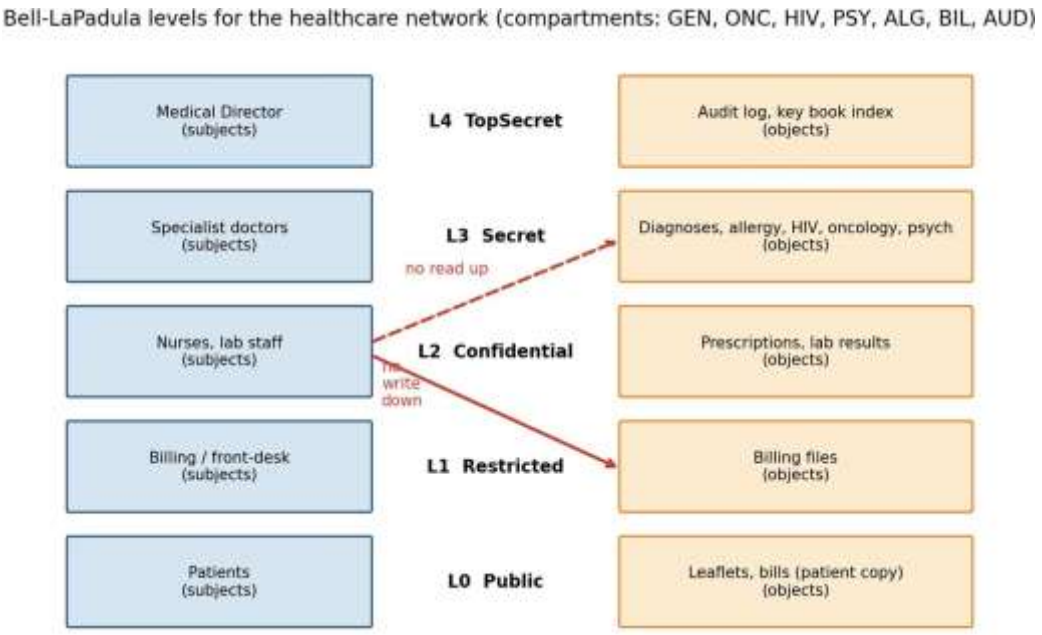


Figure 6: Bell-LaPadula levels, subjects and objects for the healthcare network.

Table 13: Levels assigned to personnel

Level	Personnel (subjects)	Compartments held
L4 Top Secret	Medical Director (Dr Rao)	GEN, ONC, HIV, PSY, ALG, BIL, AUD (all)
L3 Secret	Specialist doctors (Dr Mehta, Dr Iyer)	Mehta: GEN, ONC, HIV, ALG; Iyer: GEN, PSY, ALG
L2 Confidential	Nurses (Kavya) and lab staff (Ravi)	GEN
L1 Restricted	Billing clerks (Anita) and front-desk staff	BIL
L0 Public	Patients (P204)	none
Trusted subjects	BillingGateway, RecordsOfficer	Exempt from the star property; every use is logged

Table 14: Classification of objects

Object	Label	Reason
Leaflet	(Public, {})	Health education material
BillingFile-P204	(Restricted, {BIL})	Charges and payments contain no diagnosis
LabResult-P204	(Confidential, {GEN})	Values are clinical but not stigmatising
Prescription-P204	(Confidential, {GEN})	Needed by nurses and pharmacy to dispense
AllergyRecord-P204	(Secret, {GEN, ALG})	Reveals chronic conditions; classified Secret by the network policy
HIVDiagnosis-P311	(Secret, {HIV})	Highly stigmatising: separate compartment
OncologyReport-P311	(Secret, {ONC})	Cancer diagnosis
PsychNote-P118	(Secret, {PSY})	Mental-health note
Encounter-P311	(Secret, {BIL, HIV})	Visit record linking billing and diagnosis

Object	Label	Reason
AuditLog	(Top Secret, all)	Records every break-glass and declassification event

Compartments serve the need-to-know principle inside a level. Two Secret-cleared specialists are not equivalent: Dr Iyer is a Secret-cleared psychiatrist and must not read HIV diagnoses merely because her level equals Dr Mehta's. In BLP terms the labels (Secret, {GEN, PSY, ALG}) and (Secret, {HIV}) are incomparable, neither dominates the other, and access in both directions is denied.

c. Twenty worked access decisions

A reference monitor implementing the rules above (BLP.decide in cryptolab.py) was run on twenty requests. The table gives the decision; the commentary below explains the reasoning for each category. Eight of the twenty requests are denied and one is permitted only through the break-glass procedure.

Table 15: Access decisions made by the monitor

No	Subject	Op	Object	Decision	Rule applied
1	NurseKavya	read	LabResult-P204	PERMIT	ss-property: current level dominates object label
2	NurseKavya	read	AllergyRecord-P204	DENY	ss-property (no read up): current level does not dominate object label
3	NurseKavya	read (EMERGENCY)	AllergyRecord-P204	PERMIT	break-glass: time-limited read, mandatory audit entry + review by Director
4	DrMehta	read	HIVDiagnosis-P311	PERMIT	ss-property: current level dominates object label
5	DrIyer	read	HIVDiagnosis-P311	DENY	ss-property: level suffices but compartment(s) HIV missing (need-to-know)
6	DrMehta	read	OncologyReport-P311	PERMIT	ss-property: current level dominates object label
7	DrMehta	write	Prescription-P204	DENY	*-property (no write down): object is below the subject's current level
8	DrMehta@session-Conf	write	Prescription-P204	PERMIT	*-property: object label dominates subject's current level
9	DrMehta@session-Conf	read	HIVDiagnosis-P311	DENY	ss-property (no read up): current level does not dominate object label
10	DrMehta	write	BillingFile-P204	DENY	*-property (no write down): object is below the subject's current level
11	BillingGateway (trusted)	write	BillingFile-P204	PERMIT	trusted subject: exempt from *-property, action logged
12	BillClerkAnita	read	LabResult-P204	DENY	ss-property (no read up): current level does not dominate object label
13	BillClerkAnita	write	Encounter-P311	PERMIT	*-property: object label dominates subject's current level
14	PatientP204	read	Leaflet	PERMIT	ss-property: current level dominates object label

No	Subject	Op	Object	Decision	Rule applied
15	PatientP204	read	LabResult-P204	DENY	ss-property (no read up): current level does not dominate object label
16	RecordsOfficer (trusted)	declassify	LabResult-P204	PERMIT	trusted declassifier: tranquility relaxed, two-person rule + audit
17	NurseKavya	declassify	LabResult-P204	DENY	only trusted subjects may change an object's label (tranquility)
18	DrRao	read	AuditLog	PERMIT	ss-property: current level dominates object label
19	DrMehta	read	AuditLog	DENY	ss-property (no read up): current level does not dominate object label
20	DrMehta	read	PsychNote-P118	DENY	ss-property: level suffices but compartment(s) PSY missing (need-to-know)

Reads that are permitted (ss-property satisfied)

R01 - Nurse Kavya reads LabResult-P204. The subject label (Confidential, {GEN}) dominates the object label (Confidential, {GEN}): the levels are equal and the compartment sets are equal. Permitted. R04 and R06 - Dr Mehta reads the HIV and oncology records. His Secret clearance includes both compartments HIV and ONC, so (Secret, {GEN, ONC, HIV, ALG}) dominates (Secret, {HIV}) and (Secret, {ONC}). R14 - the patient reads the leaflet and R18 - the Director reads the audit log follow the same pattern.

Reads that must be denied (no read up, no cross-compartment)

R02 - Nurse Kavya reads the Secret allergy record: DENY. Her level (Confidential, level 2) is below the object's level (Secret, level 3), and she lacks the ALG compartment. The ss-property forbids reading up. R05 - Dr Iyer reads the HIV diagnosis: DENY. Her level equals the object's (Secret) but she does not hold the HIV compartment; the monitor reports "level suffices but compartment HIV missing". This is the compartment, or need-to-know, check. R12 - the billing clerk reads a lab result: DENY (Restricted below Confidential). R15 - the patient reads his own lab result: DENY, which exposes a policy problem discussed in Section 4.5: patients have a legal right to their own data, so the strict model must be supplemented by a release procedure (R16). R19 - Dr Mehta reads the Top Secret audit log: DENY, correctly, since an audit log that the audited party could read would be a security failure. R20 - Dr Mehta reads a psychiatric note: DENY, since PSY is not in his compartment set.

Writes (star property)

R07 - Dr Mehta, logged in at his full Secret level, writes a prescription (Confidential): DENY (no write down). This is the classic BLP result: a Secret-cleared subject could copy Secret information into a lower-level object. R08 - the same doctor in a Confidential working session writes the prescription: PERMIT. BLP allows a subject to operate at any level at or below his

clearance. When he lowers his current level to Confidential, the prescription (Confidential) dominates his current label, so the write is allowed; the price is that in this session he cannot read the HIV record (R09: DENY). R10 - Dr Mehta writes to the billing file (Restricted): DENY. R11 - the BillingGateway trusted subject writes to the billing file: PERMIT, because trusted subjects are exempt from the star property; its job is to extract only the billing code and amount from a clinical record, a controlled downgrade.

R13 - the billing clerk (Restricted, {BIL}) writes to Encounter-P311 (Secret, {BIL, HIV}): PERMIT. This is legal under BLP (writing up is allowed) yet intuitively dangerous: an untrusted, low-clearance clerk may overwrite or corrupt a Secret record without being able to read it. It is the first demonstration of the model's lack of integrity guarantees (Section 4.6).

Declassification and break-glass

R16 - RecordsOfficer declassifies a sanitised summary of the lab result to Public: PERMIT. The change of an object's label violates tranquility, so only a trusted subject may do it, with two-person approval and an audit entry. R17 - the nurse tries the same: DENY. R03 - Nurse Kavya reads the Secret allergy record in an emergency: PERMIT under break-glass. This request is the subject of the next section. The monitor appended both R03 and R16 to the audit log (two entries), which is how a later review can distinguish legitimate emergency overrides from abuse.

d. Critical evaluation in a medical setting

Emergency: a nurse needs a Secret-level allergy record

A strict BLP system answers R02 with DENY. In the scenario of an unconscious patient in anaphylaxis, the denial can cost a life: the nurse does not know whether the patient is allergic to the antibiotic about to be administered. This is the best-known mismatch between BLP and clinical practice. Confidentiality is designed to prevent disclosure, whereas emergency care requires fast disclosure.

The accepted solution is a break-glass mechanism, implemented here as request R03: (i) the nurse asserts an emergency and gives a reason; (ii) the monitor grants a time-limited read of the specific record (not a permanent promotion); (iii) every such access is written to the Top-Secret audit log and (iv) reviewed by the Director within 24 hours. The model itself has no concept of this; break-glass is an extension that deliberately violates the ss-property and substitutes accountability for prevention. The risk is abuse by curious staff: the countermeasures are the audit trail, patient notification and disciplinary consequences.

A doctor who must write a note to a lower-level billing file

The star property forbids it (R10). Clinically, the doctor must be able to record that a consultation took place so that it can be billed. Three solutions exist, and the network uses the first two. (a) A trusted downgrader (BillingGateway, R11) extracts a sanitised billing line (procedure code, fee) from the clinical record; only the gateway code must be trusted and verified. (b) The doctor works in a lower session (R08): by lowering his current level to Restricted for the billing task, the write becomes legal, but he cannot then read Secret material. (c) The doctor writes the billing information into a Secret-level object that the gateway later reads. Option (a) is the most practical but concentrates trust in one piece of software, which must be minimal, formally reviewed and logged.

Tranquility and trusted subjects

Strong tranquility says labels never change. In a hospital this is unrealistic: a diagnosis that was initially treated as Secret is later released to the patient, a new HIV result is classified at creation, and a patient's record is promoted when a cancer is suspected. R16 shows the pragmatic answer: weak tranquility, where labels change only through a trusted declassifier under audit. The danger is that every trusted subject is an exception to the model's guarantee: the proof that BLP prevents leakage assumes that no trusted subject leaks. The more exceptions are added, the more the model's assurance is replaced by faith in the trusted code.

e. Limitations of the model

Table 16: Limitations of Bell-LaPadula in healthcare

Limitation	Explanation	Healthcare example
No integrity	The model forbids only illegal disclosure; it says nothing about illegal modification. Blind write-up is explicitly allowed.	R13: a billing clerk may overwrite a Secret encounter record. A prescription at a low level may be changed by any lower-level subject (write up is allowed).
Covert channels	Information can flow through side effects the model does not control: timing, storage-full errors, file existence.	A low-level user notices that "patient 311 is locked" or that a lookup is slow, and learns that a Secret record exists. The mere existence of an oncology file leaks.
Rigidity in emergencies	The model has no notion of urgency, context or purpose.	Anaphylaxis (R02/R03): safety needs read-up, security forbids it.
Static labels	Real clearances and classifications change; strong tranquility cannot cope.	A patient moving from "routine" to "oncology" status; a nurse temporarily covering for a specialist.
Patient rights	The model has no concept of the patient as owner of the data.	R15: the patient cannot read his own result without an out-of-model release.
Needs trusted code	Exceptions concentrate all assurance in a few trusted subjects.	BillingGateway and RecordsOfficer are single points of failure.

Limitation	Explanation	Healthcare example
Does not address crypto	BLP is enforced by a reference monitor; once data leaves the monitor (a copied ciphertext on the public network), the model has no authority.	Task 12: an insider who holds the key bypasses the monitor entirely.

f. Verdict: is Bell-LaPadula alone suitable for healthcare?

No. BLP is an excellent starting point for the confidentiality requirement and gives a clear, formal answer to "who may read what", the precise question posed by the second bullet of the scenario. It maps naturally onto the five clearance levels, and compartments express need-to-know for sensitive conditions. But the scenario lists three further requirements that BLP cannot meet: detecting tampering (integrity), working under emergency conditions (flexibility), and working in rural clinics with basic hardware (a reference monitor on every terminal is a heavy demand).

The recommended policy (Chapter 12) therefore combines BLP with (i) the Biba integrity model or an equivalent rule set ("no write up" for critical objects such as prescriptions, so that low-level staff cannot alter them), (ii) a break-glass extension with mandatory audit, (iii) role-based access control for the dozens of specific tasks such as billing and pharmacy dispensing, and (iv) a cryptographic enforcement layer in which each level or compartment has its own key, so that the policy is enforced even when data is copied outside the monitor (Task 12). BLP is necessary but not sufficient.

g. Formal statement and the Basic Security Theorem

The model can be stated precisely, which is its main strength. A state is a triple (b, M, f) where b is the set of current accesses (subject, object, mode), M is the access-permission matrix and f assigns each subject a maximum and a current label and each object a classification. A state is secure if it satisfies the three properties. The Basic Security Theorem of Bell and LaPadula states: if the initial state is secure and every transition rule preserves the ss-, - and ds-properties, then every reachable state is secure.* Its proof is an induction over transitions: each rule (get-read, get-write, release, change-label) is checked once, and then the invariant holds forever.

$$read(s, o)\ allowed \iff level(s) \geq level(o) \ \wedge \ comps(s) \supseteq comps(o) \ \wedge \ o \in ACL(s)$$

$$write(s, o)\ allowed \iff level(o) \geq level(s) \ \wedge \ comps(o) \supseteq comps(s) \ \wedge \ o \in ACL(s)$$

The theorem is valuable because it replaces testing every possible sequence of actions by checking a handful of rules. Its limit is equally important: it speaks only about the security

property as defined. The theorem says nothing about integrity, availability or timing, and it assumes that all transitions are mediated. A transition performed outside the reference monitor, such as copying a ciphertext on the wire or reading a file from a stolen disk, is outside the theorem. This is why the next chapters protect data cryptographically and by policy.

h. Implementation of the reference monitor

The monitor is a small class so that it can be audited easily, a property that matters because BLP's assurance rests on the correctness of the mediating code. The core decision procedure is reproduced below; the full listing is in Appendix A. Notice that the whole read, write and declassify logic fits in about forty lines and that the audit log is written inside the same function that grants the exceptional access, so an exceptional grant cannot occur without a log entry.

```
class BLP:
    """Reference monitor: ss-property, *-property, ds-property (ACL), trusted subjects, declassification, break-glass."""

    def __init__(self):
        self.subjects, self.objects, self.acl, self.audit = {}, {}, {}, []

    def add_subject(self, name, maxlabel, current=None, trusted=False):
        self.subjects[name] = dict(max=maxlabel, cur=current or maxlabel, trusted=trusted)

    def add_object(self, name, label, acl=None):
        self.objects[name] = label
        self.acl[name] = acl

    def decide(self, s, op, o, emergency=False, reason=""):
        S, lab = self.subjects[s], self.objects[o]
        acl = self.acl[o]
        if acl is not None and s not in acl and not emergency:
            return False, "DENY  ds-property: subject is not on the object's need-to-know ACL"
        if op == "read":
            if S["cur"].dominates(lab):
                return True, "ALLOW ss-property: current level dominates object label"
            if emergency:
                self.audit.append((s, op, o, reason))
                return True, "ALLOW break-glass: time-limited read, mandatory audit entry + review by Director"
            miss = lab.comps - S["cur"].comps
            if LEVELS[S["cur"].level] >= LEVELS[lab.level] and miss:
                return False, "DENY  ss-property: level suffices but compartment(s) " + ",".join(sorted(miss)) + " missing (need-to-know)"
            return False, "DENY  ss-property (no read up): current level does not dominate object label"
        if op == "write":
            if S["trusted"]:
                return True, "ALLOW trusted subject: exempt from *-property, action logged"
            if lab.dominates(S["cur"]):
                return True, "ALLOW *-property: object label dominates subject's current level"
            if LEVELS[lab.level] >= LEVELS[S["cur"].level]:
                return False, "DENY  *-property: object label lacks compartment(s) " + ",".join(sorted(S["cur"].comps - lab.comps)) + " of the
subject"
            return False, "DENY  *-property (no write down): object is below the subject's current level"
        if op == "declassify":
            if S["trusted"]:
                self.audit.append((s, op, o, reason))
                return True, "ALLOW trusted declassifier: tranquility relaxed, two-person rule + audit"
            return False, "DENY  only trusted subjects may change an object's label (tranquility)"
        raise ValueError(op)
```

i. Break-glass procedure in operational terms

Table 17: Operational break-glass procedure (clinic, 24 hours a day)

Step	Actor	Action	Control
1	Nurse	Selects "Emergency access" and types a reason (free text, minimum 10 characters)	Reason is stored with the request
2	Monitor	Grants read of the single named record for 30 minutes; no write permission	Time limit; no label change

Step	Actor	Action	Control
3	Monitor	Appends (subject, object, reason, time) to the Top Secret audit log; alerts the Director's terminal	Entry cannot be deleted by the nurse
4	Director	Reviews within 24 hours; marks access justified or unjustified	Two-person oversight
5	Hospital	Notifies the patient when an access to a Secret record is made by someone outside the treating team	Deterrence and transparency
6	Hospital	Unjustified accesses trigger disciplinary procedure and a review of that user's privileges	Accountability replaces prevention

The procedure shows the correct way to think about break-glass: not as a hole in the model, but as a managed exception in which the loss of prevention is paid for with detection, deterrence and accountability. In a rural clinic where the nearest specialist is hours away, a rule that stops the nurse from reading the allergy record is more dangerous than the risk of occasional curiosity, and the audit trail makes curiosity expensive.

Evidence for Task 3
Twenty worked access decisions, nine denials or conditional cases explained in Section 4.3, a critical discussion of break-glass, downgrading, tranquility and trusted subjects (Section 4.4), a table of limitations and the verdict in Section 4.6.

5 Task 4 - Design the Cryptographic Strategies

a. Selection of schemes and why these five

The assignment requires at least three meaningfully different classical schemes. Five were chosen so that every family of classical technique is represented and so that each one teaches a different lesson in the later attacks. "Meaningfully different" is interpreted structurally: the schemes differ in whether they change symbols or positions, in whether the key acts on one symbol or on a block, and in whether the mapping is linear.

Table 18: The five schemes and the reason each is in the comparison

Scheme	Family	Key	Why it is included
Caesar	Monoalphabetic, additive (shift)	one shift k	Baseline. Smallest key space; shows what happens when everything is guessable and the algebra is just addition mod 26.
Monoalphabetic substitution	Monoalphabetic, arbitrary permutation	permutation of 26 letters	The key space is astronomically large (26!), so it tests the claim "large key space means strong".
Vigenère	Polyalphabetic	keyword of m letters	Flattens letter frequencies; introduces key repetition, the weakness exploited by Kasiski and the index of coincidence.
Hill (3×3)	Polygraphic, linear algebra	invertible 3×3 matrix mod m	Operates on blocks, so it hides single-letter and digram frequencies; its linearity is the weakness exploited by a known-plaintext attack. Also the only scheme that extends cleanly to digits (mod 36) for patient IDs and dosages.
Columnar transposition	Transposition	keyword giving a column order	Changes positions not symbols, so letter frequencies are preserved; the only scheme that diffuses structure; combined with substitution it gives a product cipher (Task 7).

A symmetric pair of limitations frames the comparison: substitution ciphers hide which letter is which but not where letters are, and transposition ciphers hide where but not which. Hill is the first scheme that blends both effects inside a block. No scheme here is asymmetric: the assignment statement lists "symmetric and asymmetric ciphers" but the tasks use classical (pre-modern) techniques only, and the only classical-era idea of separating the encryption and decryption keys, which is the Hill matrix and its inverse, still uses a single shared secret.

b. The worked-example message

All five worked examples use the same 41-letter message taken from the scenario (a prescription):

PRESCRIBEAMOXICILLINFIVEHUNDREDMILLIGRAMS

Non-letters are removed and letters upper-cased, the usual convention for classical ciphers. Section 5.8 shows how digits and punctuation are handled when they cannot be removed (patient IDs, dosages).

c. Caesar cipher

Algorithm and key

The key is an integer k with $1 \le k \le 25$ (k = 0 is the identity). Letters are numbered A = 0, ..., Z = 25.

$$\textit{Encryption: } C_i = (P_i + k) \bmod 26 \qquad \textit{Decryption: } P_i = (C_i - k) \bmod 26$$

```
function caesar_encrypt(text, k):
    for each letter p in text:
        output  alphabet[(index(p) + k) mod 26]

function caesar_decrypt(text, k):
    return caesar_encrypt(text, -k)
```

Worked example (k = 11)

The cipher alphabet is the plain alphabet shifted by eleven: A→L, B→M, ..., P→A. For example P (15) becomes (15 + 11) mod 26 = 0 = A and R (17) becomes 28 mod 26 = 2 = C, so PR... becomes AC... . The full ciphertext is ACPDNCTMPLXZITNTWWTYQTGPSFYOCPOXTWWTRCLXD. Decryption subtracts 11 and returns the original message exactly.

Justification

Caesar is the lower bound of the comparison. With only 25 keys it is broken by hand in minutes, and its inclusion lets the report show quantitatively how much each more elaborate scheme buys over it.

d. Monoalphabetic substitution

Algorithm and key

The key is a permutation π of the alphabet, written as a 26-letter string whose i-th letter is the image of the i-th plain letter. Encryption applies π letter by letter; decryption applies π^{-1}.

$$C_i = \pi(P_i) \qquad P_i = \pi^{-1}(C_i)$$

```
function mono_encrypt(text, key):       # key = 26-letter permutation
    table = {alphabet[i] -> key[i]}
    return [table[p] for p in text]

function mono_decrypt(text, key):
    table = {key[i] -> alphabet[i]}
    return [table[c] for c in text]
```

Worked example

The key from random seed 2024 is the cipher alphabet TQCKBAMOEDVLWUYZHRINGJSXFP (below A B C ... Z). Thus P→Z, R→R, E→B, S→I, and the whole message becomes ZRBICREQBTWYXECELLEUAEJBOGUKRBKWELLEMRTWI. The structure of the plaintext is untouched: the doubled LL in "ILLINFIVE" and "MILLIGRAMS" remains a doubled pair of the same cipher letter (the two L's in the ciphertext), and the five-times-repeated pattern of the letter I remains visible. These surviving patterns are the cryptanalyst's footholds.

Justification

Monoalphabetic substitution is the standard counter-example to "a large key space is strong". Its 26! keys (4.03×10^26) would take a hundred billion years to enumerate at a million keys per second, yet Task 6 breaks it in a fraction of a second.

e. Vigenère cipher

Algorithm and key

The key is a word K = K_0 K_1 ... K_{m−1} of m letters. The i-th plaintext letter is shifted by the (i mod m)-th key letter, so the cipher is a sequence of m interleaved Caesar ciphers.

$$C_i = (P_i + K_{i \bmod m}) \bmod 26 \qquad P_i = (C_i - K_{i \bmod m}) \bmod 26$$

```
function vigenere_encrypt(text, key):
    m = length(key)
    for i, p in enumerate(text):
        output  alphabet[(index(p) + index(key[i mod m])) mod 26]

function vigenere_decrypt(text, key):
    same with minus instead of plus
```

Worked example (key QZMKTWBRLXHD, m = 12)

The message is cut into blocks of twelve and each block is combined with the key. The first block PRESCRIBEAMO with key QZMKTWBRLXHD gives FQQCVNJSPXTR: P (15) + Q (16) = 31 mod 26 = 5 = F; R (17) + Z (25) = 42 mod 26 = 16 = Q; E (4) + M (12) = 16 = Q, and so on. The complete ciphertext is FQQCVNJSPXTRNHOSEHJEQFCHXTZNKAEDTISLWQMWL. Note that the doubled letters of the plaintext are no longer doubled and the five I's of the plaintext are encrypted to five different letters.

Justification

Vigenère was called "le chiffre indéchiffrable" for three centuries. It is included because it is the cleanest example of defeating frequency analysis by spreading each plaintext letter over m

ciphertext letters, and because the repetition of its key (period m) is the weakness that Kasiski and Friedman turned into the first general method of breaking polyalphabetic ciphers.

f. Columnar transposition

Algorithm and key

The key is a keyword of n distinct letters. The plaintext is written row by row into a grid with n columns (padded with X to fill the last row). The columns are then read out in the alphabetical order of the key letters. Only the order of the columns is secret; the key space is the number of orderings, n!.

```
function columnar_encrypt(text, key):
    n = length(key);  pad text with "X" to a multiple of n
    order = columns sorted by key letter (ties by position)
    for col in order:
        output  text[col], text[col + n], text[col + 2n], ...

function columnar_decrypt(ct, key, original_length):
    r = length(ct) / n;  split ct into n blocks of r letters
    assign block k to the k-th column in sorted order
    read the grid row by row; drop the padding
```

Worked example (key MEDICAL)

The key letters M E D I C A L sort to A C D E I L M, giving the column reading order 6, 5, 3, 2, 4, 7, 1 (the numbers 7 4 3 5 2 1 6 are the ranks of the key letters). Writing the 41-letter message into seven columns requires six rows with one X as padding. Reading the columns in order gives RXNNLSCOIUIMEALEDRREIVEGSMLHMAIIFDLXPBCIRI. All 41 letters are the same as in the plaintext; only their order differs.

Justification

Transposition is the only scheme here that does not change the identity of any letter. It is included for three reasons: it defeats digram and trigram analysis; it is the natural cipher for tabular data such as laboratory results (Section 8.4); and it combines with substitution to give the product cipher that Shannon's later theory of confusion and diffusion explains.

g. Hill cipher

Algorithm and key

The key is an n×n matrix K with entries in Z_m that is invertible modulo m (Section 6.3). The plaintext is divided into blocks of n letters (padded with X), each block is a column vector P, and the ciphertext block is C = K P mod m. Decryption multiplies by the inverse matrix.

$$C = K \cdot P \pmod{m} \qquad P = K^{-1} \cdot C \pmod{m}$$

```
function hill_encrypt(text, K, m):
    pad text with "X" to a multiple of n
    for each block P of n letters:
        C = (K * P) mod m
```

```
    output the letters of C

function hill_decrypt(ct, K, m):
  Kinv = adjugate(K) * inverse(det(K) mod m) mod m   # requires gcd(det, m) = 1
  return hill_encrypt(ct, Kinv, m)
```

Worked example (3×3 over Z26)

The key is K = [[6, 24, 1], [13, 16, 10], [20, 17, 15]] with det K = 441 ≡ 25 (mod 26), which is coprime to 26, so K is invertible. The inverse is K^{-1} = [[8, 5, 10], [21, 8, 21], [21, 12, 8]]. The first block PRE is the vector (15, 17, 4); K·P = (6·15 + 24·17 + 1·4, 13·15 + 16·17 + 10·4, 20·15 + 17·17 + 15·4) = (502, 507, 649) ≡ (8, 13, 25) = INZ. The complete ciphertext is INZRUZYEDQUYUFCLASBYOKMFPVZONTCZUAJHCMTHYH; the two repeated I's in "ILLINFIVE" are now encrypted to unrelated symbols because the encryption of a letter depends on its two neighbours.

Justification

Hill is the one classical scheme in the list that is linear, and linearity is both its attraction (it can be analysed with matrix algebra and hides digram statistics) and its weakness (n known blocks reveal the key). It also generalises naturally to the 36-symbol alphabet needed for digits.

h. Extending the alphabet to digits: Hill over Z36 for IDs and dosages

Patient identifiers such as PT-20481 and dosages such as AMOX 500MG contain digits. The alphabet is extended to 36 symbols: A-Z are 0-25 and 0-9 are 26-35. Separators (the hyphen and the space) carry no information in a fixed-format field, so they are removed before encryption and restored by the field template after decryption; the canonical forms are PT20481 and AMOX500MG. Fields whose length is not a multiple of three are padded with X. The 3×3 key is K36 = [[5,7,6],[9,5,9],[2,7,8]].

Table 19: Hill-36 encryption of structured fields

Field	Canonical	Blocks (padded)	Ciphertext	Round trip
PT-20481	PT20481	PT2 048 1XX	QO1EG4CZD	PASS
AMOX 500MG	AMOX500MG	AMO X50 0MG	YGQUUD8YE	PASS
RX-77310-B	RX77310B	RX7 731 0BX	MZ10BRXO1	PASS

Two design remarks follow. First, mod 36 is a composite modulus (36 = 4·9), so a random matrix is invertible only if its determinant is coprime to both 2 and 3. Section 6.3 shows that only 18.7% of random 3×3 matrices are valid, against 30.1% mod 26 and 97% mod a prime such as 37 or 41; a deployment that needs symbols as well as digits should use a prime modulus. Second, the blocks of an ID such as PT20481 are always the same for a given ID, so the same

patient yields the same ciphertext on every visit, which leaks that the same patient attended again. A per-message nonce prepended to the plaintext removes this (Chapter 13).

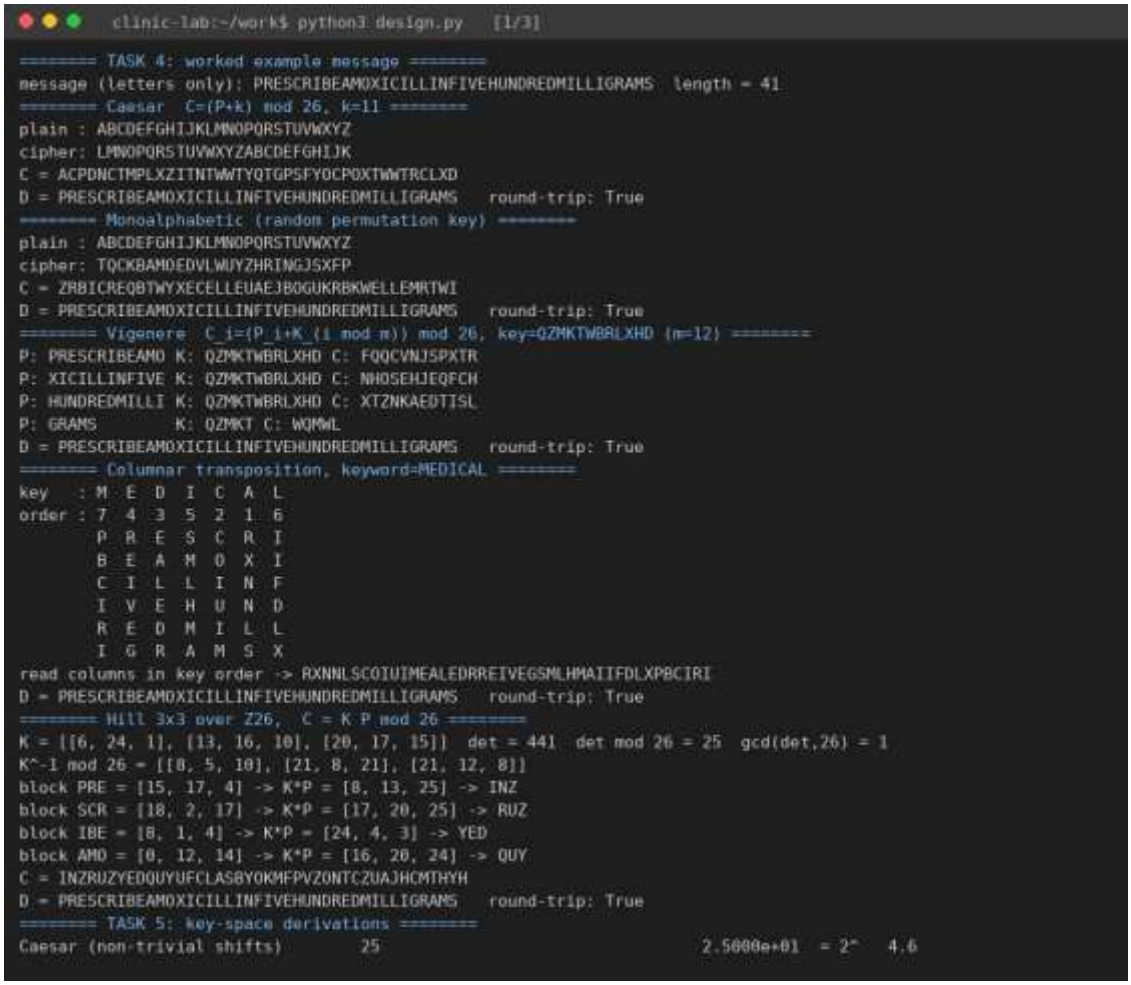


Figure 7: Execution output of s1_design.py: worked examples for all five schemes, key spaces, GL counts and the Hill-36 inverse and known-plaintext recovery (screen 1)

```
clinic-lab:~/work$ python3 design.py   [2/3]

Monoalphabetic                  26!                                 4.0329e+26  = 2^  88.4
Vigenere m=4                    26^4                                4.5698e+05  = 2^  18.8
Vigenere m=8                    26^8                                2.0883e+11  = 2^  37.6
Vigenere m=12                   26^12                               9.5429e+16  = 2^  56.4
Vigenere m=16                   26^16                               4.3609e+22  = 2^  75.2
Columnar n=7 columns            7!                                  5.0400e+03  = 2^  12.3
Columnar n=10 columns           10!                                 3.6288e+06  = 2^  21.8
Hill 2x2 mod 26                 |GL2(Z26)|                         1.5725e+05  = 2^  17.3
Hill 3x3 mod 26                 |GL3(Z26)|                         1.6340e+12  = 2^  40.6
Hill 3x3 mod 36                 |GL3(Z36)|                         1.9016e+13  = 2^  44.1
Mono + Columnar(7) product      26! x 7!                            2.0326e+30  = 2^ 100.7
Image shift mod 256             256                                 2.5600e+02  = 2^   8.0
Image pixel permutation 512x512 (512*512)!                          log2 = 4340408 bits
    (theoretical)
======== Verification of GL_n counts by exhaustive enumeration (n=2) ========
m=26: brute force = 157248   formula = 157248   match = True
m=36: brute force = 373248   formula = 373248   match = True
======== Fraction of random 3x3 matrices that are invertible, by modulus ========
mod 26: |GL3|=1.6340e+12  of 5.4295e+12  -> 30.10% invertible
mod 36: |GL3|=1.9016e+13  of 1.0156e+14  -> 18.72% invertible
mod 37: |GL3|=1.2635e+14  of 1.2996e+14  -> 97.22% invertible
mod 41: |GL3|=3.1920e+14  of 3.2738e+14  -> 97.50% invertible
======== Unicity distance U = H(K)/D, D = log2(26) - H_L, H_L ~ 1.5 bit/letter  =>  D = 3.2 ========
Caesar            H(K)=   4.6 bits  U =   1.5 letters
Mono              H(K)=  88.4 bits  U =  27.6 letters
Vigenere m=12     H(K)=  56.4 bits  U =  17.6 letters
Columnar n=7      H(K)=  12.3 bits  U =   3.8 letters
Hill 3x3 mod 26   H(K)=  40.6 bits  U =  12.7 letters
Mono+Col7         H(K)= 100.7 bits  U =  31.5 letters
======== Hill 3x3 over Z36 (alphabet A-Z,0-9)  -- invertibility proof and inverse by hand ========
K = [[5, 7, 6], [9, 5, 9], [2, 7, 8]]
det(K) = -175   det mod 36 = 5   gcd(det,36) = 1 -> invertible
det^-1 mod 36 = 29   (check 5*29 mod 36 = 1)
cofactor matrix C = [[-23, -54, 53], [-14, 28, -21], [33, 9, -38]]
adjugate = C^T    = [[-23, -14, 33], [-54, 28, 9], [53, -21, -38]]
K^-1 = det^-1 * adj mod 36 = [[17, 26, 21], [18, 20, 9], [25, 3, 14]]
K * K^-1 mod 36 = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]  (identity)
======== Encrypting structured fields (separators removed, restored by field template) ========
PT-20481     -> canonical PT20481    -> padded blocks ['PT2', '048', '1XX'] -> C = QO1EG4CZD
    decrypt ok: True
```

Figure 8: Execution output of s1_design.py: worked examples for all five schemes, key spaces, GL counts and the Hill-36 inverse and known-plaintext recovery (screen 2)

```
clinic-lab:~/work$ python3 design.py   [3/3]

AMOX 500MG    -> canonical AMOX500MG  -> padded blocks ['AMO', 'X50', '0MG'] -> C = YGQUUD8YE
    decrypt ok: True
RX-77310-B    -> canonical RX77310B   -> padded blocks ['RX7', '731', '0BX'] -> C = MZ10BRXO1
    decrypt ok: True
======== Known-plaintext attack on Hill-36 using the predictable header PATIENTID-DIAGNOSIS-
    PRESCRIPTION ========
known pairs (plain block -> cipher block): [('PAT', 'JSC'), ('IEN', 'C3E'), ('TID', 'ZWK'), ('DIA',
    '950')] ...
triples of known blocks: 120;  plaintext matrix invertible mod 36 in 31 (25.8%)
first three header blocks PAT,IEN,TID -> P = [[15, 8, 19], [0, 4, 8], [19, 13, 3]]  det mod 36 = 12
    gcd = 12
blocks used: (0, 1, 4)  recovered K = [[5, 7, 6], [9, 5, 9], [2, 7, 8]]
RECOVERED KEY EQUALS TRUE KEY: True
======== Hill 2x2 mod 26 known-plaintext toy example ========
K = [[3, 3], [2, 5]]  det = 9  mod 26 = 9   P=HELP -> C = HIAT
P matrix = [[7, 11], [4, 15]]  C matrix = [[7, 8], [8, 19]]  K = C P^-1 = [[3, 3], [2, 5]]  match:
    True
```

Figure 9: Execution output of s1_design.py: worked examples for all five schemes, key spaces, GL counts and the Hill-36 inverse and known-plaintext recovery (screen 3)

i. Encrypting clinical notes of 200 to 10,000 characters

A synthetic clinical-note generator produced letters-only notes of 200, 1,000, 5,000 and 10,000 characters (about 100 distinct sentences compose each). Each note was encrypted by Caesar (k = 11), monoalphabetic substitution, Vigenère (12 letters), columnar transposition (MEDICAL), Hill 3×3 (K26) and the product mono∘columnar. The first 60 ciphertext letters of the 1,000-letter note illustrate the effect of each cipher; the plaintext begins LABORATORYINVESTIGATIONSHOWEDTHATTHEPLATELETCOUNTWASMILDL YRA.

Table 20: First 60 letters of each ciphertext (1,000-letter note)

Scheme	Ciphertext
Caesar	WLMZCLEZCJTYGPDETRLETZYDSZHPOESLEESPAWLEPWPENZFYEHLDXTWOWJCL
Monoalphabetic	LTQYRTNYRFEUJBINEMTNEYUIOYSBKNOTNNOBZLTNBLBNCYGUNSTIWELKLFRT
Vigenere	BZNYKWUFCVPQLDEDBCBKTLUVXNIOWPIREQOHFKMDXHFKNLBQJVMCFEMUWVYD
Columnar	AVTWTENLSVEDPRDEJENELUTOIGHAIEONETTPTRAAOWDMEOTBEATSELVIIPPE
Hill 3x3	PXBYMXZRPLYFGXEATIWUBYYRKPGLEAJVJAJNQHTNFPOMRFIJSBAOMITPUGMP
Mono+Columnar	TJNSNBULIJBKZRKBDBUBLGNYEMOTEBYUBNNZNRTTYSKWBYNQBTNIBLJEEZZB

Visually, Caesar and monoalphabetic ciphertexts look random but a trained eye sees the repeated groups; Vigenère and Hill look uniformly random; the columnar ciphertext looks like scrambled English because it contains exactly the plaintext letters. Whether these impressions mean anything is the subject of Chapters 7 and 8.

Evidence for Task 4
Five distinct schemes (Caesar, monoalphabetic, Vigenère, Hill, columnar), each with an algorithm, key definition, a worked example on the same 41-letter prescription and a justification (Sections 5.3-5.7), plus the Hill-36 extension to patient IDs and dosages (Section 5.8) and encryption of notes up to 10,000 characters (Section 5.9).

6 Task 5 - Mathematical Analysis of the Schemes

a. Encryption as a modular function

Every scheme in this report is a function on integers modulo m, which is why the algebra of Z_m governs its security. The table gives each encryption in that form together with its inverse and the condition on the key that makes the inverse exist.

Table 21: Modular form of each scheme

Scheme	Encryption	Decryption	Key condition
Caesar	C = (P + k) mod 26	P = (C − k) mod 26	any k
Affine variant	C = (aP + b) mod 26	P = a^{-1}(C − b) mod 26	gcd(a, 26) = 1 (12 values of a)
Monoalphabetic	C = π(P), π a permutation of Z26	P = π^{-1}(C)	π bijective
Vigenère	C_i = (P_i + K_{i mod m}) mod 26	P_i = (C_i − K_{i mod m}) mod 26	any key word
Hill (block n)	C = K P mod m, K ∈ M_n(Z_m)	P = K^{-1} C mod m	gcd(det K mod m, m) = 1
Columnar transposition	C_j = P_{σ(j)}, σ a permutation of positions	P_i = C_{σ^{-1}(i)}	σ derived from n distinct key letters
Image shift	C = (P + k) mod 256	P = (C − k) mod 256	any k
Image permutation	C_j = P_{σ(j)}, σ a permutation of the N pixel positions	P_i = C_{σ^{-1}(i)}	σ bijective

Caesar is the affine cipher with a = 1; Vigenère is a periodic sequence of Caesar shifts; Hill is the n-dimensional generalisation of the affine cipher with the scalar a replaced by a matrix K and b = 0. The homogeneous form (b = 0) matters for the cryptanalysis: a Hill cipher maps the zero vector to itself, and K P_1 + K P_2 = K (P_1 + P_2), the linearity exploited in Section 7.5.

b. Key-space derivations

Caesar

There are 26 shifts, one of which (k = 0) is the identity, so the useful key space is 25 = 2^4.6. Counting k = 0 gives 26.

Monoalphabetic substitution

A key is a bijection from 26 plaintext letters to 26 ciphertext letters. The first letter has 26 images, the second 25, and so on:

$$|K| = 26 \times 25 \times 24 \times \ldots \times 1 = 26! = 4.0329 \times 10^{26} = 2^{88.4}$$

If one wishes to exclude keys with fixed points (letters mapped to themselves), the count is the number of derangements, 26!/e ≈ 1.48×10^26, which changes the bit length by only 1.4 bits.

Vigenère

Each of the m key letters can be chosen independently from 26, so |K| = 26^m, which grows by 4.7 bits per extra key letter. For the 12-letter key used here, 26^12 = 9.54×10^16 = 2^56.4; for a 16-letter key, 2^75.2. If the key length is not fixed, the key space is the sum over all lengths m ≤ M, which is dominated by its largest term.

Columnar transposition

A key of n distinct letters determines a permutation of n columns, so |K| = n!: 7! = 5,040 (12.3 bits) for MEDICAL, 10! = 3,628,800 (21.8 bits) for ten columns. The key space of a transposition grows more slowly than that of a substitution; even 26 columns give only 26! = 2^88.4, but a 26-column key is impractical for a nurse to remember or to use by hand.

Hill cipher: counting invertible matrices

The number of valid keys is not m^(n²) but the order of the general linear group GL_n(Z_m). Two facts reduce the count to prime fields. (i) By the Chinese Remainder Theorem, Z_m ≅ Z_{p_1^{e_1}} × ... , so a matrix is invertible mod m exactly when it is invertible modulo every prime power dividing m, and |GL_n(Z_m)| is the product of the counts for each prime power. (ii) For a prime power p^e, a matrix is invertible exactly when its reduction mod p is invertible, so |GL_n(Z_{p^e})| = p^{(e−1)n²} · |GL_n(F_p)|. For a prime field the rows must be linearly independent: the first row can be any of p^n − 1 non-zero vectors, the second any of p^n − p vectors not a multiple of the first, the third any of p^n − p² vectors outside the span of the first two, and so on:

$$|GL_n(F_p)| = (p^n - 1)(p^n - p)(p^n - p^2) \dots (p^n - p^{n-1})$$

For n = 3 and m = 26 = 2·13 this gives $|GL_3(F_2)| = 7 \cdot 6 \cdot 4 = 168$ and $|GL_3(F_{13})| = (2197-1)(2197-13)(2197-169) = 2196 \cdot 2184 \cdot 2028 = 9{,}726{,}417{,}792$, and the product 168 × 9,726,417,792 is 1.634×10^12 = 2^40.6. For m = 36 = 4·9 the factors are 2^9·168 = 86,016 for 4 and 3^9·11,232 = 221,079,456 for 9, and the product is 1.902×10^13 = 2^44.1. For n = 2 the same formula gives 157,248 mod 26 and 373,248 mod 36; both counts were verified by exhaustive enumeration of all 26^4 and 36^4 matrices (Table below).

Table 22: Key-space summary

Scheme	Formula	Size	Bits
Caesar (non-trivial shifts)	25	25	4.6
Monoalphabetic	26!	4.03×10^26	88.4
Vigenere m=4	26^4	456,976	18.8
Vigenere m=8	26^8	2.09×10^11	37.6

Scheme	Formula	Size	Bits
Vigenere m=12	26^12	9.54×10^16	56.4
Vigenere m=16	26^16	4.36×10^22	75.2
Columnar n=7 columns	7!	5,040	12.3
Columnar n=10 columns	10!	3.63×10^6	21.8
Hill 2x2 mod 26	|GL2(Z26)|	157,248	17.3
Hill 3x3 mod 26	|GL3(Z26)|	1.63×10^12	40.6
Hill 3x3 mod 36	|GL3(Z36)|	1.90×10^13	44.1
Mono + Columnar(7) product	26! x 7!	2.03×10^30	100.7
Image shift mod 256	256	256	8.0
Image pixel permutation 512x512	(512*512)!	≈ 10^(1.3×10^6)	4340408.5

Table 23: Verification of the GL_2 counts by exhaustive enumeration

Modulus	Matrices tested	Invertible (brute force)	Formula	Match
26	$26^4 = 456{,}976$	157,248	157,248	PASS
36	$36^4 = 1{,}679{,}616$	373,248	373,248	PASS

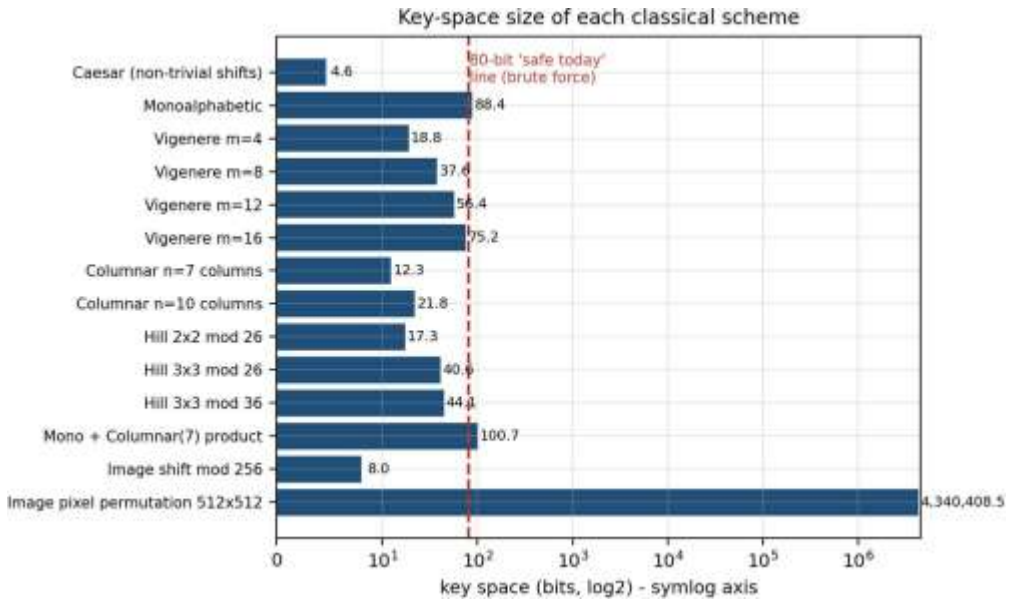


Figure 10: Key-space size of every scheme in bits (log scale). The dashed line marks 80 bits.

c. Hill cipher: when does K^{-1} mod m exist?

Theorem. Let K be an n×n integer matrix and m ≥ 2. Then K has an inverse modulo m if and only if gcd(det K, m) = 1.

Proof. (⇐) Let d = det K and suppose gcd(d, m) = 1. Then d has a multiplicative inverse d^{-1} modulo m (extended Euclid). The adjugate matrix adj(K), the transpose of the cofactor matrix, satisfies K · adj(K) = adj(K) · K = d · I over the integers. Multiplying by d^{-1} gives K · (d^{-1} adj

K) ≡ I (mod m). (⇒) Suppose K L ≡ I (mod m). Taking determinants, det K · det L ≡ 1 (mod m), so det K is a unit modulo m, which means gcd(det K, m) = 1. ■

For m = 26 the condition is "det K is odd and not a multiple of 13"; for m = 36 it is "det K is not divisible by 2 or 3". A random 3×3 matrix satisfies it with probability $|GL_3(Z_m)| / m^9$:

Table 24: Probability that a random 3×3 matrix is a valid Hill key

Modulus m	Invertible matrices	All matrices m^9	Fraction
26	1.63×10^12	5.43×10^12	30.10%
36	1.90×10^13	1.02×10^14	18.72%
37	1.26×10^14	1.30×10^14	97.22%
41	3.19×10^14	3.27×10^14	97.50%

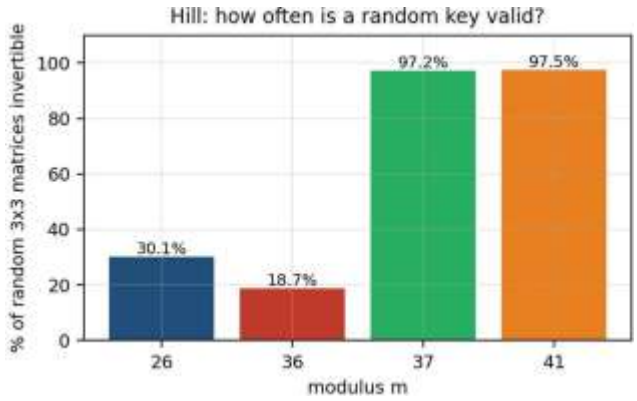


Figure 11: Fraction of random keys that are valid Hill keys: prime moduli are far friendlier.

The consequence for the design is practical. A clinic that generates a key by trial must try on average 3.3 matrices (mod 26) or 5.3 matrices (mod 36), but only 1.03 matrices if a prime modulus such as 37 or 41 is used. The 36-symbol alphabet of Section 5.8 is therefore a pragmatic choice, not an optimal one.

d. Computing one inverse by hand

Over Z26

For K = [[6,24,1],[13,16,10],[20,17,15]]:

det K = 6(16·15 − 10·17) − 24(13·15 − 10·20) + 1(13·17 − 16·20)

= 6·70 − 24·(−5) + (−99) = 420 + 120 − 99

= 441.

Since 441 = 16·26 + 25, det K ≡ 25 ≡ −1 (mod 26) and gcd(25, 26) = 1, so K is invertible with $\det^{-1} = 25$ (because 25·25 = 625 = 24·26 + 1).

The cofactors and adjugate then give $K^{-1} = [[8, 5, 10], [21, 8, 21], [21, 12, 8]]$ (verified by the program as $K \cdot K^{-1} = I$).

Over Z36 (the key used for IDs and dosages)

For K36 = [[5,7,6],[9,5,9],[2,7,8]]:

$\det K = 5(5 \cdot 8 - 9 \cdot 7) - 7(9 \cdot 8 - 9 \cdot 2) + 6(9 \cdot 7 - 5 \cdot 2)$

$= 5(-23) - 7(54) + 6(53)$

$= -115 - 378 + 318$

$= -175.$

Modulo 36 this is 5 (since $-175 + 180 = 5$), and $\gcd(5, 36) = 1$, so the key is invertible.

The inverse of 5 modulo 36 is 29 because $5 \times 29 = 145 = 4 \cdot 36 + 1$.

The cofactor matrix is C = [[-23,-54,53],[-14,28,-21],[33,9,-38]] and its transpose, the adjugate, is [[-23,-14,33],[-54,28,9],[53,-21,-38]]. Multiplying the adjugate by 29 and reducing modulo 36 gives K^{-1} = [[17,26,21],[18,20,9],[25,3,14]]. As a check, $K \cdot K^{-1}$ mod 36 equals the identity matrix (printed by the program in Section 5.8's execution screen). Decrypting the field ciphertext QO1EG4CZD with this matrix returns PT20481XX, confirming the inverse.

A 2×2 toy example for the known-plaintext attack

With K = [[3,3],[2,5]] (det 9, gcd(9, 26) = 1), the plaintext HELP encrypts to HIAT. Writing the plaintext blocks HE and LP as columns gives P = [[7,11],[4,15]] and the ciphertext blocks as C = [[7,0],[8,19]]. Then $K = C\ P^{-1} \bmod 26$ = [[3,3],[2,5]], recovered exactly. The same algebra, with 3×3 blocks and a longer header, breaks the real key in Section 7.5.

e. Does a large key space guarantee strength?

No. A large key space is a necessary condition against exhaustive search, not a sufficient condition for security. The monoalphabetic cipher is the counter-example:

Table 25: Key space versus real resistance

Scheme	Key bits	Exhaustive search at 10^9 keys/s	Actual attack	Time to break
Caesar	4.6	< 1 s	chi-square over 26 shifts	< 1 ms
Monoalphabetic	88.4	6.4×10^9 years	hill climbing on quadgram fitness	0.07 s for 1,000 letters
Vigenère m = 12	56.4	1.5 years	IC + per-column chi-square	≈ 0.1 s
Columnar n = 7	12.3	< 1 s	brute force of 5,040 orders	0.06 s
Hill 3×3 mod 26	40.6	13.6 minutes	known-plaintext, linear algebra	< 1 ms

Four independent reasons explain why a huge key space does not save the monoalphabetic cipher.

i. **The key space is not searched uniformly.** An attacker does not enumerate 26! permutations; he climbs a hill. Each swap of two letters is scored by how English-like the result looks, and the score of the true key is a global maximum that the search reaches in a few hundred evaluations.

ii. **Letter frequencies survive the cipher.** A substitution is a relabelling: the sorted frequency profile of the ciphertext is *identical* to that of the plaintext (Figure in Chapter 7), and the index of coincidence of the ciphertext is exactly that of English (0.0695 in the experiment). The key changes the names of the letters, not the statistics.

iii. **Natural language has very high redundancy.** The unicity distance of the monoalphabetic cipher is only U = H(K)/D = 88.4 / 3.2 ≈ 28 letters (table below): beyond about 28 letters of ciphertext only one key makes the text meaningful, so the key is *information-theoretically* determined by a few lines of text, and the remaining question is only computational.

iv. **The effective key is far smaller than the nominal key.** Only the 8 to 10 most frequent letters of a typical note determine most of the text; the remaining letters are fixed almost automatically by context.

Table 26: Unicity distance (redundancy D = 3.2 bits per letter)

Scheme	H(K) bits	Unicity distance U (letters)
Caesar	4.6	1.5
Mono	88.4	27.6
Vigenere m=12	56.4	17.6
Columnar n=7	12.3	3.8
Hill 3x3 mod 26	40.6	12.7
Mono+Col7	100.7	31.5

The bound U measures when the ciphertext contains enough information to fix the key; it does not say the attack is easy. For Vigenère, U = 17.6 letters but the attack needs about 400-800 letters because the attacker must also find the key by a search procedure. The gap between U and the observed attack lengths in Chapter 7 is the "computational slack" that classical ciphers have always relied upon, and that modern computing has removed.

7 Task 6 - Evaluate Cryptanalytic Resistance

Each scheme is attacked with a technique chosen to exploit the specific property that the scheme leaves exposed. The attacker is assumed to know the algorithm (Kerckhoffs's principle) and to hold a public reference corpus of English and clinical text. All attacks were run on the same 1,000-letter note unless stated, and the success rates of Section 7.8 use twelve random keys per length.

Table 27: Attack matched to scheme

Scheme	Property exploited	Attack used
Caesar	Letter frequency preserved; 25 keys	Brute force + chi-square
Monoalphabetic	Letter frequency and digram structure preserved	Frequency-rank guess, then hill climbing with quadgram fitness
Vigenère	Key repeats with period m	Kasiski examination, index of coincidence, per-column chi-square
Columnar	Letter frequency preserved; few orders	Brute force over column orders scored by quadgram fitness
Hill	Linearity of encryption	Known-plaintext: $K = C\,P^{-1}$; digram statistics for diagnosis
Product (mono∘columnar)	Both parts keep partial structure	Digram-IC to undo transposition, then hill climbing

a. Evidence from letter frequencies and the index of coincidence

Before any attack, the statistics of each ciphertext already tell the cryptanalyst what kind of cipher he faces. The index of coincidence (IC) is the probability that two randomly chosen letters of the text are equal; English has IC ≈ 0.066-0.070 (a clinical-note vocabulary is slightly higher), uniform random letters 0.0385.

Table 28: Index of coincidence by scheme and message length

Length	Plain	Caesar	Mono	Vigenère	Columnar	Hill 3×3	Mono+Col
200	0.0695	0.0695	0.0695	0.0381	0.0676	0.0379	0.0676
1000	0.0695	0.0695	0.0695	0.0406	0.0694	0.0387	0.0694
5000	0.0700	0.0700	0.0700	0.0405	0.0698	0.0388	0.0698
10000	0.0698	0.0698	0.0698	0.0405	0.0697	0.0390	0.0697

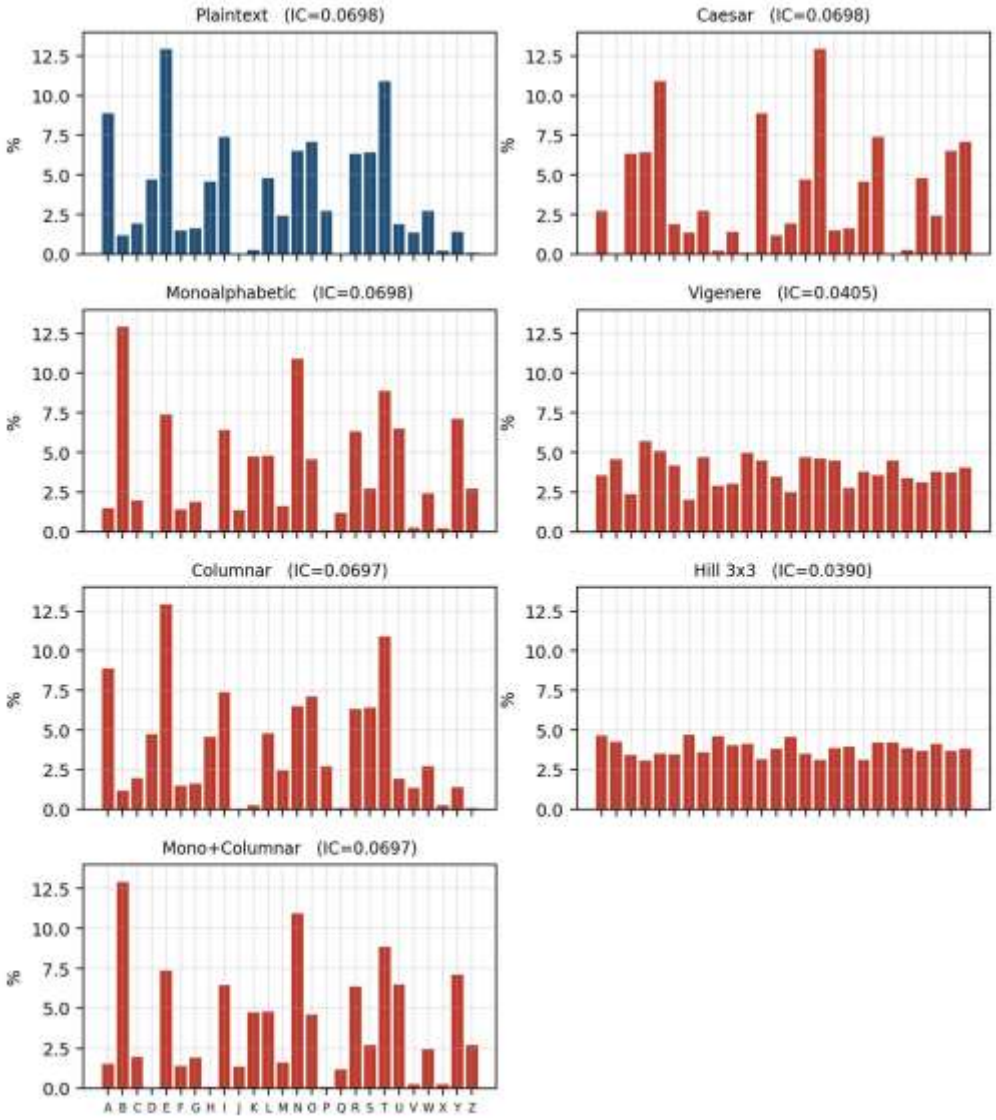


Figure 12: Letter-frequency histograms of a 10,000-letter note and of its ciphertexts under each scheme.

Four patterns are visible. Caesar, monoalphabetic, columnar and the product cipher have the same IC as the plaintext (0.0698 for 10,000 letters) because they only relabel or reorder letters, so they leave the multiset of letter counts unchanged. Vigenère drops to 0.0405 and Hill to 0.0390, close to the random value 0.0385: both cipher a given letter to many different letters. The sorted profiles in the next figure show the same thing with the labels removed: five of the six ciphertexts have exactly the plaintext's profile in different order, and only Vigenère and Hill are flat.

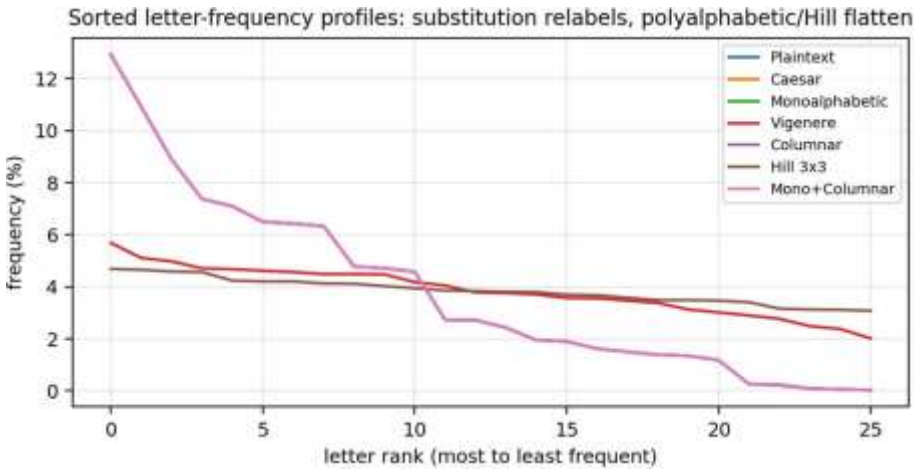


Figure 13: Sorted letter-frequency profiles: substitution and transposition keep the plaintext profile; Vigenère and Hill flatten it.

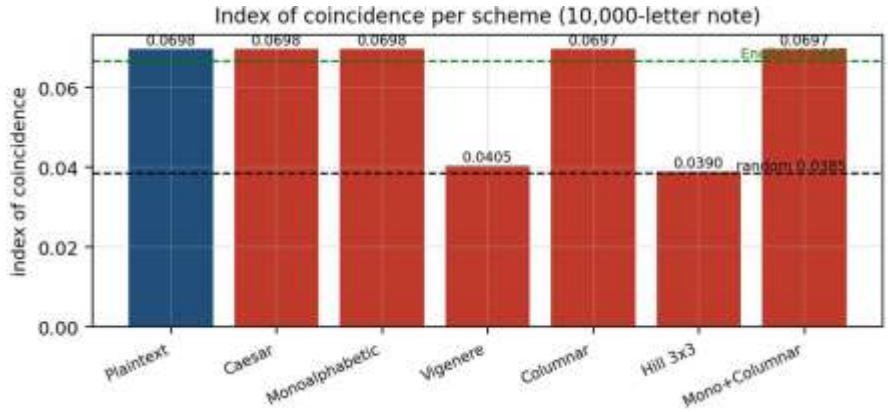


Figure 14: Index of coincidence per scheme (10,000 letters).

b. Attack on Caesar: brute force with chi-square

The cryptanalyst tries all 26 shifts and scores each candidate by the chi-square distance between its letter counts and English letter frequencies. The correct shift 11 has chi-square 49.0, whereas the next best shift scores 3915, a gap of nearly two orders of magnitude (log scale below), and the candidate plaintext begins LABORATORYINVESTIGATION. The attack needs no more than 50 letters; with 50 letters 12 of 12 random keys were recovered.

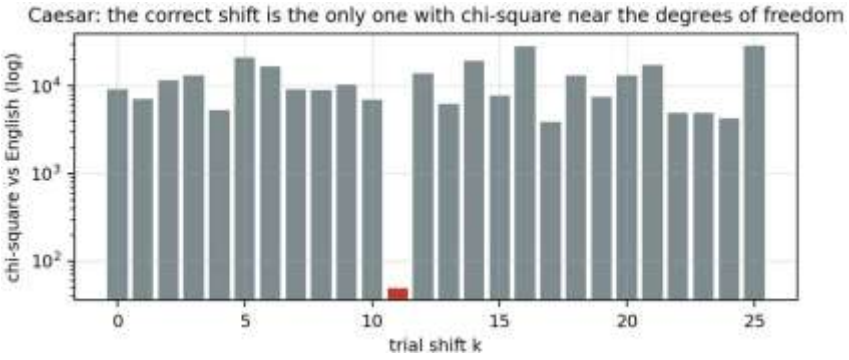


Figure 15: Caesar: chi-square against English for each of the 26 shifts.

Why it succeeds: a Caesar shift maps the whole frequency profile of English one place to the right, so a single parameter (the shift) explains the entire ciphertext, and the plaintext is the only candidate whose profile matches English.

c. Attack on the monoalphabetic cipher: frequency analysis and hill climbing

Step 1: frequency ranking. Ranking the ciphertext letters by frequency and aligning them with the English order E T A O I N S H R D L ... yields a first guess with only 44.9% of the letters correct. Frequency counting is not enough alone because the order of the middle-frequency letters of a 1,000-letter text fluctuates.

Step 2: hill climbing. The attacker starts from the frequency guess and repeatedly swaps two letters of the key, keeping the swap if the log-likelihood of the resulting plaintext under a quadgram model of English improves. Restarts from perturbed keys avoid local maxima. Using the quadgram table built from the reference corpus, the attack recovered 100.0% of the letters in 0.07 seconds.

```
cipher  : LTQYRTNYRFEUJBINEMTNEYUIOYSBKNOTNNOBZLTNBLBNCYGUNSTIWELKLFRTEIBKYURBJE
naive   : DABNSATNSWIOVEHTIFATINOHRNUELTRATTRECDATEDETGNYOTUAHMIDLDWSAIHELNOSEVI
climbed : LABORATORYINVESTIGATIONSHOWEDTHATTHEPLATELETCOUNTWASMILDLYRAISEDONREVI
plain   : LABORATORYINVESTIGATIONSHOWEDTHATTHEPLATELETCOUNTWASMILDLYRAISEDONREVI
```

Why it succeeds: (i) a monoalphabetic cipher preserves single-letter frequencies, which gives a starting point; (ii) it preserves digram and quadgram structure up to relabelling, so the quadgram fitness is a smooth landscape with the true key as the global maximum; (iii) the fitness of a trial key is cheap to compute (a few thousand additions), making hundreds of thousands of trials affordable. The attack's success depends on the amount of ciphertext: Section 7.8 shows that 100 letters succeed half the time and 400 letters almost always.

d. Attack on Vigenère: Kasiski and the index of coincidence

A Vigenère cipher is m interleaved Caesar ciphers. To attack it the cryptanalyst first needs m, then solves m Caesar ciphers.

Kasiski examination. If the same plaintext fragment (such as THE or PATIENT) is enciphered twice at positions whose distance is a multiple of m, it produces the same ciphertext fragment. The program found 90 repeated 3-5 letter groups and counted the factors of the distances between repetitions. The factor 12 received 76 votes, together with its divisors 2, 3, 4 and 6 (which divide every multiple of 12), while unrelated factors such as 5, 7 and 11 received almost none. The vote for 12 and not 24 shows m = 12.

Index of coincidence. For each trial length m the ciphertext is split into m columns (letters m apart) and the IC of each column is averaged. If m is the true length, each column is a Caesar cipher and has the IC of English; otherwise columns mix several shifts and have IC ≈ 0.04. The measured average IC is 0.068 at m = 12, 0.054 at m = 6, 0.052 at m = 18 and 0.040-0.048 elsewhere, selecting m = 12 unambiguously.

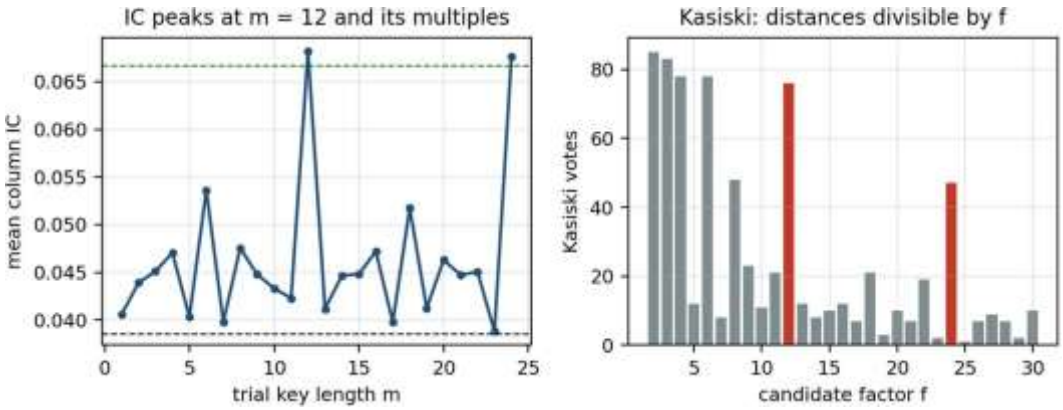


Figure 16: Vigenère key-length detection: mean column IC for trial lengths (left) and Kasiski factor votes (right).

Per-column solution. With m known, each of the 12 columns is a Caesar cipher of about 83 letters, and chi-square recovers its shift. The key recovered was QZMKTWBRLXHD, which equals the true key QZMKTWBRLXHD, and the plaintext was restored exactly.

Why it succeeds: the key repeats. Repetition makes the plaintext coincide with itself in the keystream, which Kasiski sees as repeated ciphertext groups, and it makes every m-th letter a monoalphabetic sample, which the IC sees as an English-like profile. A longer key helps only while the ciphertext is shorter than roughly 8-10 times the key length.

e. Known-plaintext attack on Hill

Because encryption is linear, C = K P, if the attacker knows n plaintext blocks and their ciphertext blocks, the key is recovered by one matrix inversion: stack the blocks as columns of P and C, then $K = C P^{-1} \pmod m$. The only requirement is that P is itself invertible modulo m. Predictable headers such as PATIENTIDDIAGNOSISPRESCRIPTION supply the known blocks.

In the 36-symbol design the plaintext header blocks PAT, IEN, TID have P = [[15,8,19],[0,4,8],[19,13,3]] whose determinant is 12 mod 36, not coprime to 36, so the first three blocks cannot be used. The program tested all 120 triples of the ten header blocks: 31 (25.8%) are invertible. Using blocks (0, 1, 4) the program recovered exactly K36 = [[5,7,6],[9,5,9],[2,7,8]] and could decrypt every message.

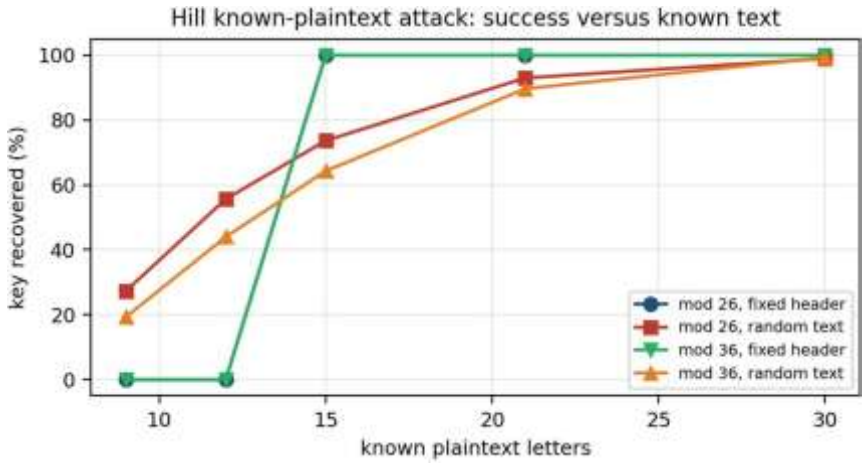


Figure 17: Hill known-plaintext attack: percentage of random keys recovered versus number of known letters.

With random known text the success probability follows the invertibility of the plaintext matrix: 27.3% of mod-26 keys and 19.3% of mod-36 keys are recovered from 9 letters, rising to 99% from 30 letters. For the actual header text, the key is recovered in 100% of cases from 15 known letters. Why it succeeds: linearity. The relation C = K P holds for every block, so n independent blocks give n^2 linear equations in the n^2 unknown entries of K, which a single matrix inversion solves.

Ciphertext-only resistance. The Hill cipher is much better against ciphertext-only attack. Its IC is 0.039 (near random), and the digram IC falls from 0.0091 (plaintext, monoalphabetic ciphertext) to 0.0021, so neither letter nor pair statistics reveal anything:

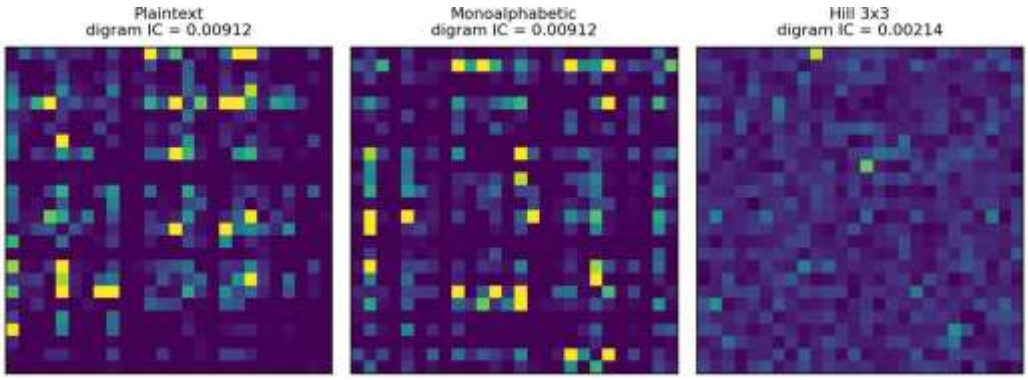


Figure 18: Digram frequency maps: plaintext and monoalphabetic ciphertext have the same concentrated structure; Hill ciphertext is flat.

A ciphertext-only attack on a 3×3 Hill cipher exists in the literature (statistics of block trigrams), but it needs thousands of letters and was not implemented here. The correct statement is therefore: Hill resists ciphertext-only frequency analysis, but is broken by a known-plaintext attack with 15 letters.

f. Attack on columnar transposition

The attacker has no letter frequency to exploit because the multiset of letters is unchanged; he instead tries every column count n (2 to 8) that divides the ciphertext length and every column order (n! of them), and scores each candidate plaintext with the quadgram model. For the 7-column key the program tested all 5,040 orders and found the true one, with 100% accuracy in 0.06 s. Why it succeeds: the key space is tiny (12.3 bits) and the correct order is the only one that produces valid English quadgrams; transposition keeps letter identities, so every correct local adjacency scores highly.

g. Attack on the product cipher: monoalphabetic ∘ columnar

A substitution and a transposition commute (one acts on symbol values, the other on positions), so the attacker may undo either first. Hill climbing cannot be applied directly because the quadgrams of the transposed ciphertext are scrambled. The program therefore first undoes the transposition using a statistic that is invariant under substitution: the digram index of coincidence, i.e. the probability that two consecutive letters coincide with another consecutive pair. Under the correct column order, the digram IC equals that of English (about 0.009); under any wrong order it falls to about 0.0015. Once the columns are in order, the text is a plain monoalphabetic cipher and Section 7.3 applies.

Table 29: Product-cipher attack (columnar key MEDICAL, one fixed substitution key)

Length	Columns found	Accuracy	Time (s)	Result
400	7	7.2%	0.09	FAIL
1000	7	100.0%	0.11	SUCCESS
3000	7	100.0%	0.19	SUCCESS

At 400 letters the attack fails (7.2% accuracy: the digram statistics of so short a text are too noisy to select the column order) but at 1,000 and 3,000 letters it succeeds completely. The product cipher therefore raises the length of ciphertext the attacker needs, but not the kind of attack that works.

h. Attack success rate versus ciphertext length

Table 30: Success rate of ciphertext-only attacks (%), 12 random keys per cell

Length	Caesar	Mono (clin.+Eng.)	Mono (Eng. only)	Vigenère 12	Columnar 7	Mono + Col
50	100	0	0	0	100	0
100	100	50	0	0	100	8
200	100	50	25	25	100	67
400	100	100	83	83	100	100
800	100	100	75	100	100	83
1600	100	100	100	100	100	100
3200	100	100	100	100	100	100

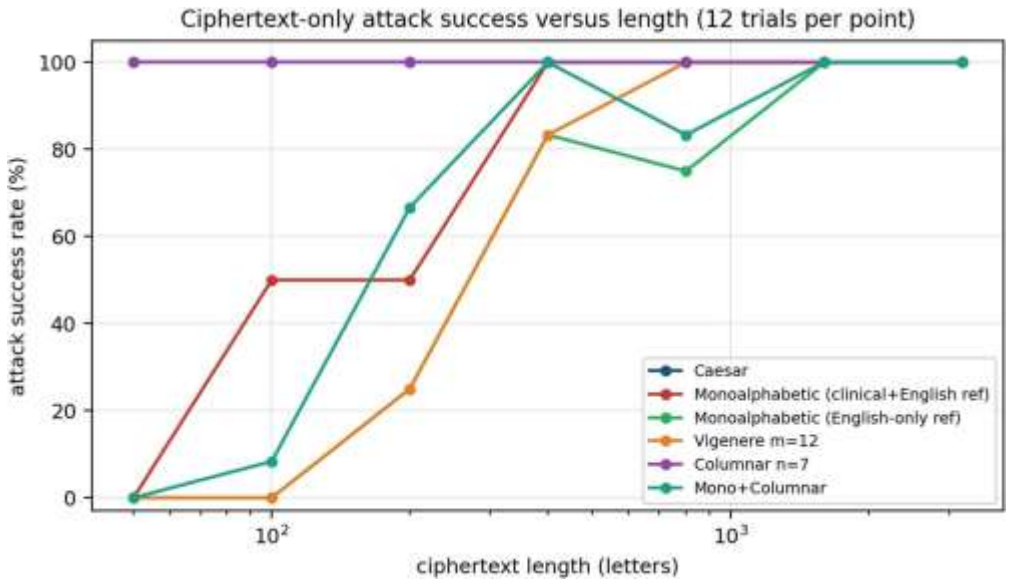


Figure 19: Attack success versus ciphertext length for each ciphertext-only attack.

The curves have the characteristic sigmoid shape of statistical attacks: below a critical length the attack fails, above it the attack almost always succeeds, and the transition is sharp. The critical length is the true measure of a classical cipher's strength and is much more informative than the key-space size. Reading the 90% crossing: Caesar needs about 50 letters, columnar 7 about 50 (given a good language model), monoalphabetic about 400 (800 if the attacker lacks domain-specific reference text), Vigenère with 12 letters about 800, and the product cipher about 400. The difference between the two monoalphabetic columns shows a practical point: an attacker with a clinical reference corpus needs fewer letters than one who has only generic English, but the gap closes by 1,600 letters.

i. Brute-force feasibility and why exhaustive search is not the right yardstick

Table 31: Time to exhaust each key space at three attacker speeds

Scheme	Bits	10^6 keys/s (laptop)	10^9 keys/s (GPU)	10^12 keys/s (cluster)
Caesar	4.6	< 1 second	< 1 second	< 1 second
Vigenere m=4	18.8	< 1 second	< 1 second	< 1 second
Vigenere m=8	37.6	1.2 days	1.7 minutes	< 1 second
Vigenere m=12	56.4	1,512.0 years	1.5 years	13.3 hours
Vigenere m=16	75.2	6.91e+08 years	690,938.8 years	690.9 years
Columnar n=7	12.3	< 1 second	< 1 second	< 1 second
Columnar n=10	21.8	1.8 seconds	< 1 second	< 1 second
Hill 3x3 mod 26	40.6	9.5 days	13.6 minutes	< 1 second
Monoalphabetic	88.4	6.39e+12 years	6.39e+09 years	6.39e+06 years
Mono+Col7	100.7	3.22e+16 years	3.22e+13 years	3.22e+10 years
AES-128 (reference)	128.0	5.39e+24 years	5.39e+21 years	5.39e+18 years

On this measure only the 16-letter Vigenère key (75 bits) and the monoalphabetic key (88 bits) look safe; yet the attacks above break the monoalphabetic cipher in a second. Brute force is the upper bound on an attacker's effort and only the lower bound on a designer's security; the real bound is the cleverest attack, which for statistical ciphers is the one that avoids enumeration entirely.

j. Ranking the schemes by resistance

Table 32: Ranking by resistance (1 = most resistant)

Rank	Scheme	Ciphertext-only: letters for ≥90% success	Known-plaintext effort	Evidence
1	Hill 3×3	no attack implemented; IC 0.039, digram IC 0.0021	15 letters of header (100%)	Section 7.5, Figure on Hill KPA
2	Vigenère (m = 12)	≈ 800	12 key letters (header of length 12 suffices)	Section 7.4, Section 13.2

Rank	Scheme	Ciphertext-only: letters for ≥90% success	Known-plaintext effort	Evidence
3	Mono + Columnar (7)	≈ 400-1,000	about 20 distinct letters + column order	Sections 7.7, 7.8
4	Monoalphabetic	≈ 400 (clinical corpus) / 800 (English only)	about 20 distinct letters	Sections 7.3, 7.8
5	Columnar (7)	≈ 50	a few header letters	Section 7.6
6	Caesar	≈ 50, trivial by hand	1 letter	Section 7.2

The ranking is not a single scale. Hill is first because it has no efficient ciphertext-only attack among those implemented, but it is the most brittle under known-plaintext: when headers are predictable it has the lowest margin of the polyalphabetic and polygraphic schemes. Vigenère is second: it demands the most ciphertext among the schemes that were actually broken, but one header is enough for key recovery. The product cipher is third: it needs somewhat more ciphertext than its parts, though much less than its 100-bit key space suggests, because the digram-IC trick separates the two layers. The ranking therefore supports a specific design rule: no classical cipher should be used alone, and no key should protect predictable text for long.

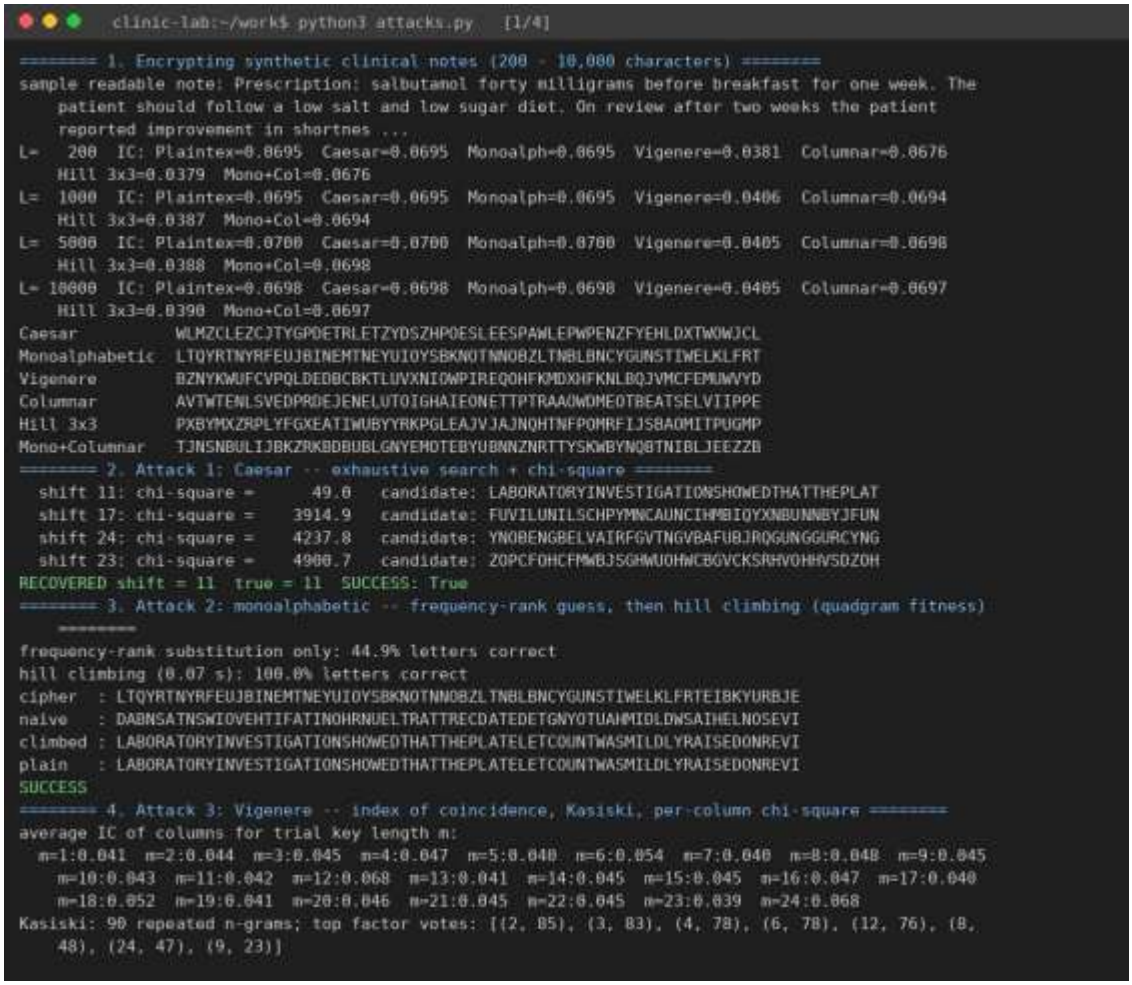


Figure 20: Execution output of s2_attacks.py: encryption of notes, every attack, success-rate experiment, Hill known-plaintext tests, digram IC, timing and brute-force tables (screen 1)

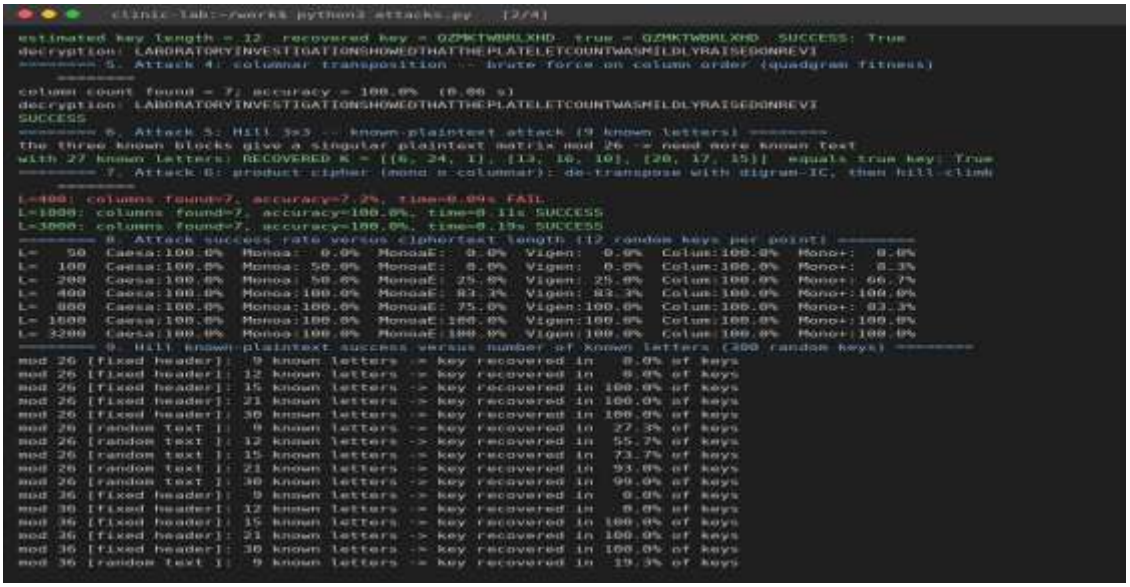


Figure 21: Execution output of s2_attacks.py: encryption of notes, every attack, success-rate experiment, Hill known-plaintext tests, digram IC, timing and brute-force tables (screen 2)

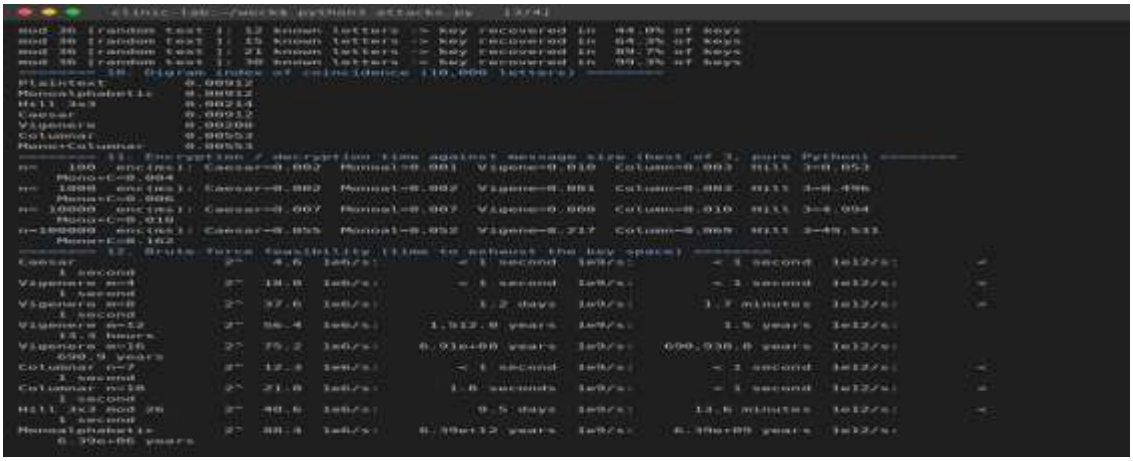


Figure 22: Execution output of s2_attacks.py: encryption of notes, every attack, success-rate experiment, Hill known-plaintext tests, digram IC, timing and brute-force tables (screen 3)



Figure 23: Execution output of s2_attacks.py: encryption of notes, every attack, success-rate experiment, Hill known-plaintext tests, digram IC, timing and brute-force tables (screen 4)

Evidence for Task 6

Six attacks, each matched to its scheme (Table of Section 7), each with the reason it succeeds in terms of frequency preservation, key repetition or linearity, plus a success-rate experiment over seven ciphertext lengths and a justified ranking (Section 7.10).

8 Task 7 - Assess Transposition and Product Ciphers

a. Implementation and how transposition diffuses structure

The columnar transposition of Section 5.6 was implemented (col_enc, col_dec) and tested on notes of 100 to 100,000 letters with decryption verified in every case (Chapter 10). The question here is what it does to the structure of text. Two measures answer it.

- **Single-letter statistics are untouched.** The IC of the columnar ciphertext of a 10,000-letter note is 0.0697 against 0.0698 for the plaintext: the multiset of letters is identical (a tiny difference arises only from the 5 X pad letters).
- **Adjacent-letter statistics are destroyed.** The digram IC drops from 0.00912 for the plaintext to 0.00553 for the columnar ciphertext, a loss of about 40% (not 100%, since letters that were 7 positions apart in the plaintext become adjacent within a column and the plaintext has some long-range correlation). Quadgrams are scrambled much more strongly.

Transposition thus provides diffusion in Shannon's sense: the influence of each plaintext position is spread over the ciphertext. For a 7-column grid with 6 rows, a letter moves by up to 5 rows and 6 columns, so letters that were adjacent are separated by roughly one column height (here about 6 positions). It provides no confusion: the relation between key and ciphertext is a plain permutation that can be undone by trial.

b. Is substitution followed by transposition stronger than either alone?

Evidence comes from three sources, all measured in this report.

i. **Letter-frequency evidence.** The IC of the product ciphertext is 0.0697, the same as plaintext: the product cipher does *not* hide single-letter frequencies, because both layers preserve the letter counts (the substitution relabels them, the transposition reorders them). Frequency evidence therefore gives no advantage over the monoalphabetic cipher alone.

ii. **Digram evidence.** The product cipher's digram IC (0.0055) matches that of the columnar cipher, not of the plaintext, so it is the transposition that hides the digrams. This is the contribution of the layer: an attacker can no longer run a quadgram fitness test directly.

iii. **Attack evidence.** The success curves of Section 7.8 show that the product cipher needs at least as much ciphertext as either component but only moderately more: 67% success at 200 letters against 50% for

the monoalphabetic alone and 100% for the transposition

alone. The two layers are *separable* (they commute), and the digram IC statistic undoes the transposition independently of the substitution key.

Verdict. The combination is stronger than either component in the narrow sense that the attacker needs a two-stage procedure and somewhat more text, and its nominal key space is the product 26!·7! = 2^100.7. But its practical strength is far below 100 bits: the layers do not interact, so the work is the sum of the work for each layer (a few thousand evaluations) rather than the product. A genuine product cipher in Shannon's sense must make the layers depend on each other, for example by using the substitution key to derive the transposition key, or by iterating several rounds; this is the principle behind the Data Encryption Standard.

c. Practicality for rural clinic staff

Table 33: Manual effort estimate per 200-character clinical note

Operation	Elementary steps	Time (assumed 2 s per step)	Error risk
Caesar (shift by key)	200 look-ups in a printed strip	≈ 7 minutes	Low
Monoalphabetic	200 look-ups in a 26-row table	≈ 7 minutes	Low-medium; table must be memorised or carried
Vigenère (12-letter key)	200 table look-ups in a 26×26 grid plus tracking the key position	≈ 10-12 minutes	Medium: slipping out of key position corrupts the rest
Columnar (7 columns)	write 200 letters in a grid, read 7 columns	≈ 13 minutes	Medium: miscounting columns
Hill 3×3	67 blocks × 9 multiplications and reductions	≈ 20-40 minutes	High: arithmetic errors propagate through a block
Mono + Columnar	sum of the two	≈ 20 minutes	Medium-high

The timings are assumptions, stated openly, but the conclusion does not depend on their exact values. A clinician writes a 200-letter note in perhaps four minutes; encrypting it by hand would take longer than writing it. Computers, even basic ones, remove the time cost entirely: the program encrypts 100,000 characters in under 0.15 seconds (Chapter 11), so the only realistic deployment is a small program on the clinic computer, in which case the "usability for untrained staff" depends on the key handling, not on the cipher. The reasoning of Chapter 12 is that the simplest workable operation for a nurse is: type a keyword once per session; the software does everything else.

d. Lab table: transposition, integrity tests and classical checksums

Laboratory results are tabular and fixed-width, which makes them a good fit for columnar transposition (each ciphertext block corresponds to a table column) but also makes tampering easy to target. A table of 24 rows was built, each row exactly 32 characters (patient ID, test

code, value, unit, flag), plus a 32-character checksum row. The 800-character body was encrypted by columnar transposition with the 8-column key HOSPITAL (each ciphertext block has 100 characters, four per row). Sample rows:

```
PT20481|GLU |  85.9|MG/DL  |N|;
PT20481|CRE |   1.4|MG/DL  |N|;
PT20481|HGB |   9.9|G/DL   |N|;
PT20481|WBC |   8.9|K/UL   |N|;
PT20482|K   |   3.6|MMOL/L |N|;
PT20482|NA  | 137.7|MMOL/L |N|;
PT20486|PLT |  397.9|K/UL  |N|;
```

The ciphertext begins 1 G|1 G|1 /|1 /|2 M|2 M|2 /|2 /|3 G|3 G|3 /|3 /|: the regular table layout is still visible in the ciphertext as repeating patterns ("1 G|1 G|1 /|1 /"), because identical field positions in successive rows land in adjacent places. This is an important weakness for tabular data and the reason that transposition alone should never protect structured records.

Test A: one changed ciphertext character

The attacker, without the key, replaced one ciphertext character. The receiver decrypted without any error and one row had changed: PT20484|K | 3.8|MMOL/L |N|; became PT20484|K | 73.8|MMOL/L |N|;, a potassium value of 3.8 became 73.8. Transposition by itself raised no alarm: detected: no. Both checksums, in contrast, failed to match (additive ok = false, Fletcher ok = false) and flagged the tampering.

Test B: swapping two whole rows

Rows 3 and 9 were swapped by exchanging the 4-character slices that hold each row in every one of the 8 ciphertext blocks, again without the key (the block length is public). After decryption row 3 and row 9 had exchanged their contents (PT20481 WBC with PT20483 CRE), producing a perfectly formatted table in which the results belong to the wrong patients. The additive checksum did not notice (ok = true) because a sum does not depend on the order of the terms; the positional Fletcher checksum did (ok = false).

The classical checksums

Table 34: Checksum designs

Checksum	Definition	Detects single edit	Detects row swap
Additive	$S = \Sigma\ c_i \bmod 997$	Yes	No
Fletcher-style	$a = \Sigma\ c_i \bmod 251$; $b = \Sigma$ (running a) mod 251 (positional)	Yes	Yes

Test C: where the attacker defeats the checksum

The Fletcher checksum is a pair of linear sums: $S_1 = \Sigma\ c_i$ and $S_2 = \Sigma\ i \cdot c_i$ (up to a fixed linear relabelling). Both are preserved by any change d_i to three consecutive characters with d = (a,

−2a, a), since Σ d_i = a − 2a + a = 0 and Σ i d_i = i·a − 2(i+1)a + (i+2)a = 0. Searching the table the program found row 13: with d = (2, -4, 2) the value 144.1 became 306.1, a change of more than a factor of two, and both checksums still matched (additive ok = true, Fletcher ok = true). The attacker needs to know which ciphertext characters correspond to which plaintext positions, which an insider with the key (or an outsider who has learned the column order from a known header) does.

Test D and keyed checksums

If the checksum is unkeyed, the attacker who can edit the table can also recompute the tag row, and the receiver's check passes: detected: no for any unkeyed checksum. A keyed classical checksum was tried: the checksum digits are enciphered with a Vigenère key. But the attacker who sees one message and its tag subtracts the public checksum from the tag to obtain the key stream (RXQMW), then forges a valid tag for any message he likes (final lines of the execution screen). Every classical keyed checksum is a linear function, and linear functions are recovered by linear algebra from a few known input-output pairs.

Lesson
Columnar transposition hides content only weakly and gives no integrity. A classical checksum detects accidental errors and clumsy edits, but is defeated by (i) order-independence (additive), (ii) compensating edits (any linear sum) and (iii) recomputation (any unkeyed or linearly keyed tag). Integrity requires a cryptographic MAC or signature, which is outside the classical toolkit.

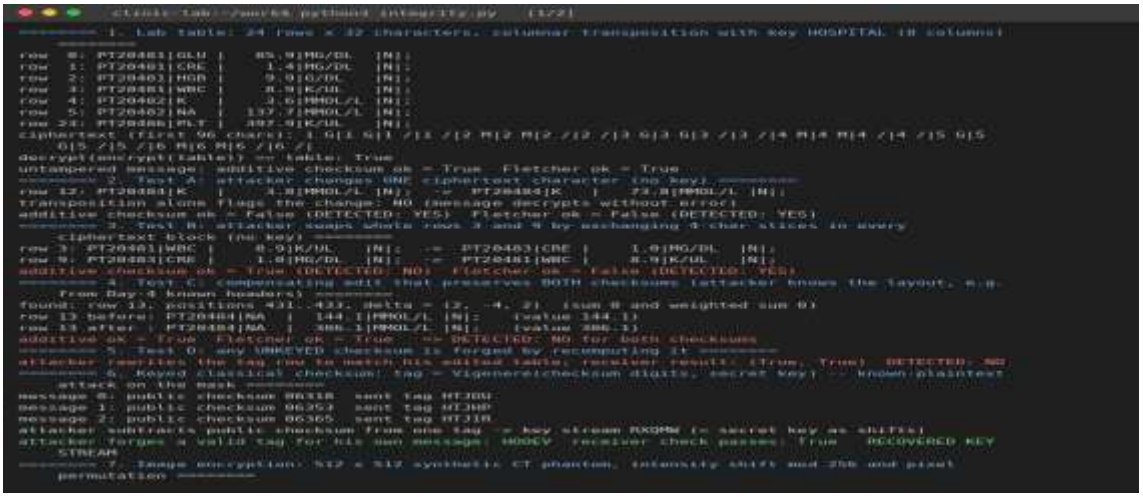


Figure 24: Execution output of s4_integrity_images.py: lab-table tamper tests, checksum forgeries and image-cipher metrics (screen 1)

```
clinic-lab:~/work$ python3 integrity.py   [2/2]

shape (512, 512), dtype uint8; decrypt(shift) == original: True  (1.83 ms)
decrypt(permutation) == original: True  (4.17 ms);  perm+shift round trip: True
Original            entropy=5.3112 bits  adjacent-pixel r=+0.9714  neighbour MI=2.1077 bits
    gradient corr. with original=+1.0000
Shift mod 256       entropy=5.3112 bits  adjacent-pixel r=+0.8755  neighbour MI=2.1081 bits
    gradient corr. with original=+1.0000
Pixel permutation   entropy=5.3112 bits  adjacent-pixel r=+0.0010  neighbour MI=0.0019 bits
    gradient corr. with original=-0.0022
Permutation + shift  entropy=5.3112 bits  adjacent-pixel r=+0.0045  neighbour MI=0.0019 bits
    gradient corr. with original=-0.0022
======== 8. Attacks on the image ciphers ========
shift: histogram spike of the black background sits at 87; assuming background = 0 -> k = 87
    SUCCESS: True
shift: one known pixel pair (p=28, c=115) -> k = 87  RECOVERED
permutation: histogram identical to original; background fraction 24.1% (leaks tissue/air ratio);
    positions unrecoverable from histogram alone
```

Figure 25: Execution output of s4_integrity_images.py: lab-table tamper tests, checksum forgeries and image-cipher metrics (screen 2)

e. Extending substitution and transposition to medical images

The two basic image operations correspond directly to the two classical families: an intensity shift mod 256, C = (P + k) mod 256 (a Caesar cipher on grey levels), and a pixel permutation, which is a transposition of all N = 512 × 512 positions under a keyed permutation. A synthetic 512×512 CT-like phantom was encrypted with k = 87 and a permutation seeded from the key 20481, and both were decrypted and compared pixel by pixel. Result: decrypt(shift) = original: true; decrypt(permutation) = original: true; the combination also round-trips (true).

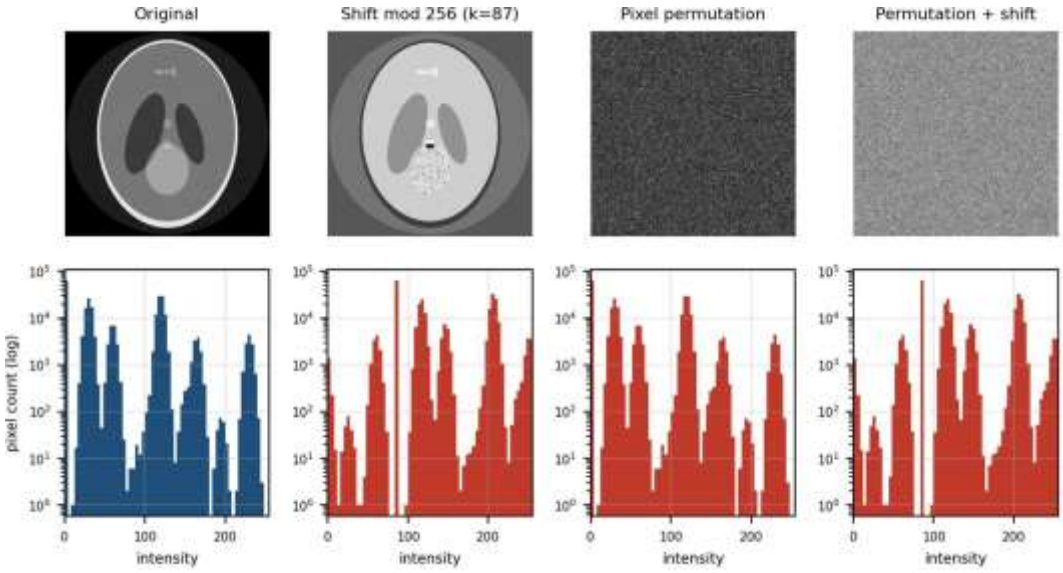


Figure 26: Original scan, intensity shift (k = 87), pixel permutation and the two combined, with their histograms (log scale).

Table 35: Image-cipher metrics

Image	Entropy (bits)	Adjacent-pixel r	Neighbour MI (bits)	Gradient corr. with original
Original	5.3112	+0.9714	2.1077	+1.0000

Image	Entropy (bits)	Adjacent-pixel r	Neighbour MI (bits)	Gradient corr. with original
Shift mod 256	5.3112	+0.8755	2.1081	+1.0000
Pixel permutation	5.3112	+0.0010	0.0019	-0.0022
Permutation + shift	5.3112	+0.0045	0.0019	-0.0022

What the histograms show

Intensity shift. The histogram is the original histogram rotated by k = 87: the dominant spike of the black background at grey level 0 now sits at level 87. The image looks different, but the shapes of the skull and brain are fully recognisable (Figure above, panel 2). The entropy 5.3112 bits is identical to that of the original, because a relabelling cannot change the entropy; the mutual information between neighbouring pixels (2.108 bits) is also unchanged, and the gradient (edge) map is perfectly correlated with the original (r = 1.000) when differences are taken modulo 256. The only visible artefacts are lines where the intensities wrap past 255. The shift cipher leaks the full structure of the image.

Pixel permutation. The histogram and entropy are unchanged (the multiset of pixels is the same), but the image looks like noise: adjacent-pixel correlation falls from +0.971 to +0.001 and neighbour mutual information from 2.108 to 0.002 bits. The spatial structure is removed. What it leaks is the histogram: anyone can read off that 24.1% of the pixels are black background and see the characteristic intensity distribution of a CT head, which discloses the image type and rough content.

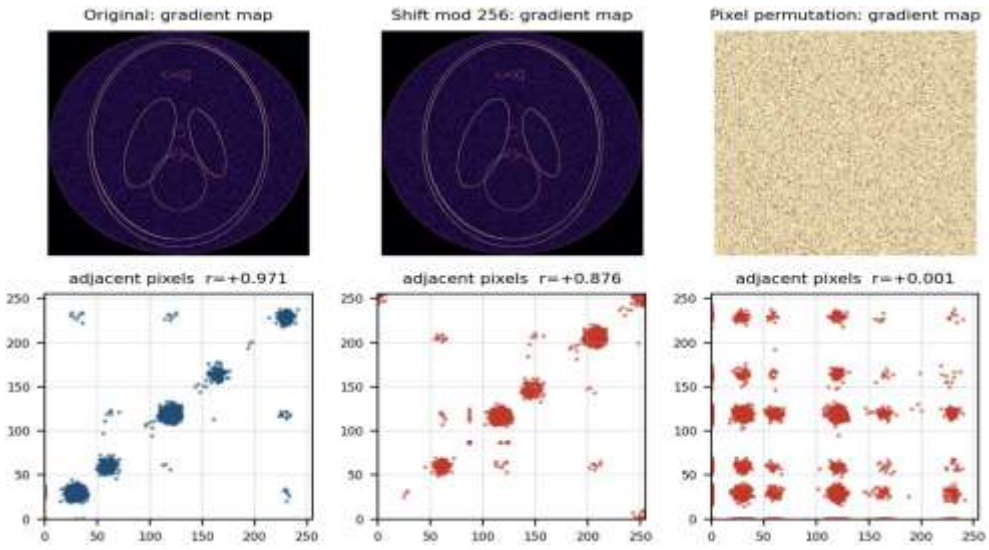


Figure 27: Top: gradient maps. Bottom: adjacent-pixel scatter plots. Shift keeps both; permutation destroys both.

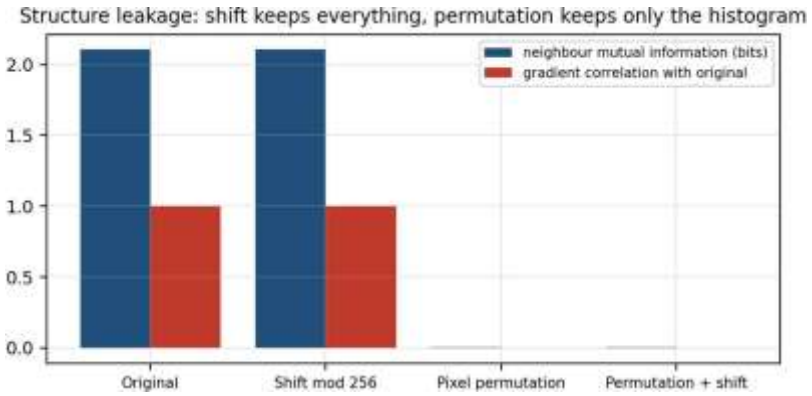


Figure 28: Structure leakage measures for each image cipher.

Attacks on the image ciphers

Shift: the attacker, seeing that the largest histogram spike is the background (black, level 0 in all scans), reads off k = 87; one known pixel pair (p = 28, c = 115) gives k = 87 directly. Permutation: a histogram reveals nothing about positions; the attack is a jigsaw puzzle solved using the strong neighbour correlation of natural images (+0.97), which is feasible for 512×512 images with known techniques but was not run here. The full-image key space is (512×512)! = 2^4.3 million bits, an absurd nominal value, because the key as implemented is a 32-bit seed, so the effective key space is only 2^32.

Which method leaks the image's structure? The intensity shift: it leaks structure completely and the histogram. The permutation leaks the histogram only. The combination (permute then shift) hides the structure and also moves the histogram, so its only leak is the shape of the histogram. For medical images, which have characteristic intensity distributions, even that residual leak is informative, and neither method gives integrity: flipping a pixel in the encrypted image flips one pixel in the decrypted image.

Evidence for Task 7
Implementation and diffusion measurement of columnar transposition (Section 8.1), a three-part evaluation of the product cipher with a verdict (Section 8.2), a practicality estimate for rural staff (Section 8.3), lab-table tests A-D with checksum defeats (Section 8.4), and image encryption with histograms, entropy and structure leakage (Section 8.5).

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Threats, Attacks and Risk Evaluation	4
Task 2 – Security Goals Mapping	3
Task 3 – Bell-LaPadula Application and Critique	5
Task 4 – Cipher Design	4
Task 5 – Mathematical Analysis	5
Task 6 – Cryptanalytic Resistance	4
Task 7 – Transposition / Product Ciphers	3
Task 8 – Steganography Evaluation	3
Task 9 – Implementation & Testing	4
Task 10 – Experimental Evidence	3
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**